

SUB-ID-744354.pdf

 Datta Meghe Institute of Medical Sciences, Wardha

Document Details

Submission ID

trn:oid:::27005:138256289

Submission Date

May 9, 2026, 9:27 AM GMT+5:30

Download Date

May 9, 2026, 10:00 AM GMT+5:30

File Name

SUB-ID-744354.pdf

File Size

1.5 MB

83 Pages

17,375 Words

101,516 Characters

44% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

-  **67 AI-generated only 44%**
Likely AI-generated text from a large-language model.
-  **0 AI-generated text that was AI-paraphrased 0%**
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



Trustline

A Self-Hosted End-to-End Encrypted Real-Time Collaboration Platform for Confidential Enterprise Workflows

ACKNOWLEDGEMENT

First, I wish to thank the Almighty, who gave me good health and success throughout my project work.

I wish to express my heartfelt thanks to our Honorable Founder, Dr. S. Jagathrakshakan, and Dr. J. Sundeeep Aanand, President, for providing me with the necessary facilities to complete my project.

I dedicate my thanks to esteemed Dr. M. Sundararajan, Vice-Chancellor In-charge, for his support and encouragement.

I express my sincere gratitude to Dr. A. Muthukumaravel, Dean PG, Faculty of Arts and Science, Dr. S. Arumugam, Dean UG, Faculty of Arts and Science, and Dr. S. Silvia Priscila, Associate Professor and Head, Department of Computer Science, Faculty of Arts and Science, for their support during the project to make it successful.

I dedicate my thanks to my beloved Guide _____, Department of Computer Science, Faculty of Arts and Science, for his/her support and valuable guidance throughout the project work.

I offer my cordial thanks to my parents, friends, staff members, and well-wishers for their continued encouragement throughout my career. I also thank people who have directly or indirectly given me encouragement and support throughout the project.

Mukesh raja (U23SS0012)

Praveen kumar (U23SS0014)

Praveen J (U23SS0015)

ABSTRACT

A lot of people use cloud-based collaboration tools these days, which has created a security paradox: these platforms can make people more productive, but they often use centralized architectures that let service providers get decryption keys without their knowledge. This “Implicit Trust” model has a big single point of failure that puts businesses at risk of “middleman” attacks and cross-tenant data leaks if the central infrastructure is hacked. This project gives you Trustline, a self-hosted collaboration platform that works very well and uses a strict client-side Zero-Knowledge Framework to keep sensitive business workflows safe. Blind Router logic is what makes up the Trustline architecture. This means that the backend relay server can only send encrypted blobs and can’t read the messages or the cryptographic keys. The system does sensitive tasks like making keys, the X3DH handshake, and Double Ratchet advancement only in the local device runtime. It was made with the Tauri v2 framework and a Rust cryptographic core that is safe for memory. This design makes sure that private keys are never sent over the network or leave the device’s OS Secure Enclave or an encrypted SQLCipher vault.

Trustline mainly uses WebAuthn/FIDO2 (Passkeys) to check who you are. This connects their identity to the biometrics on their physical hardware, which gets rid of the risks that come with using regular passwords. The Double Ratchet algorithm in the Signal Protocol makes message keys that are different for each interaction. This makes it very safe before and after a breach. At the database level, PostgreSQL Row-Level Security (RLS) also keeps tenants separate from each other. This means that the data for each organization is stored in different places, which makes it less likely that data will leak between tenants. Tests in the real world show that these strict security rules don’t make the user experience worse. Trustline’s average message synchronisation latency is 110 ms, which is much better than the 250 ms limit that is needed for smooth real-time interaction. Also, if you use Argon2id to make keys, the system will be safe from brute-force attacks for more than 112,000 years if you use hardware to speed things up. In the end, Trustline gives businesses full digital sovereignty, which means that even if the hosting infrastructure is hacked completely, their data will stay private

Keywords: End-to-End Encryption (E2EE), Zero-Knowledge Architecture, WebAuthn, Double Ratchet, Argon2id, Self-Hosting, Rust

TABLE OF CONTENTS

S.NO.	TITLE	PAGE NO.
	Abstract	vi
	Table of Contents	vii
	List of Figures	ix
1.	INTRODUCTION	1
1.1	An Overview	1
1.2	Objective & Scope of the Project	1
1.3	Literature Survey	1
1.4	Problem Statement	1
2.	SYSTEM ANALYSIS	8
2.1	Existing system	8
2.2	Disadvantages of Existing system	8
2.3	Proposed system	8
2.4	Advantages of Proposed System	8
3.	SYSTEM CONFIGURATION	12
3.1	Hardware Requirements	12
3.2	Software Requirements	12
4.	SYSTEM DESIGN	15
4.1	Overall Architecture diagram	15
4.2	Methodology / Algorithm Adopted	15
5.	SYSTEM IMPLEMENTATION	22
5.1	Module Description	22
6.	SYSTEM TESTING	28
6.1	Unit Testing	28

6.2	Integrated Testing	28
6.3	Black Box Testing	28
6.4	White Box Testing	28
6.5	Verification Testing	28
6.6	Validation Testing	28
6.7	User Acceptance Testing	28
7.	Results and Discussion	35
8.	CONCLUSION	40
9.	LIMITATIONS & SCOPE OF FUTURE WORK	42
10.	REFERENCES	44
11.	APPENDIX	45
11.1	Source Code	45
11.2	Output Screen Shots	45

LIST OF FIGURES

FIGURE NO.	DESCRIPTION
Figure 1	Zero-Knowledge “Blind Router” Logic
Figure 2	X3DH Key Agreement
Figure 3	The Double Ratchet Protocol
Figure 4	PostgreSQL Row-Level Security (RLS)
Figure 5	WebAuthn Passwordless Authentication
Figure 6	CRDT Synchronization Flow
Figure 7	Identity Verification Screen
Figure 8	Encrypted Message Vault
Figure 9	Administrator Device Portal
Figure 10	Relay Node Monitor
Figure 11	Admin Console - Pending Approvals
Figure 12	Admin Console – Policies

I. INTRODUCTION

1.1 An Overview

Traditional security models like enterprise firewalls and Virtual Private Networks (VPNs) are mostly useless against modern, advanced cyber threats because of the quick move to distributed cloud environments and the permanent move to remote workforces. Real-time collaboration is an important part of any modern business in today's digital world. But this reliance has made systems weaker by putting sensitive data in cloud ecosystems where service providers can easily get to decryption keys. This "Implicit Trust" model puts businesses at a lot of risk from "middlemen" and the chance of lateral movement if the main infrastructure is hacked.

Trustline is a high-performance, self-hosted collaboration platform that uses a strict client-side Zero-Knowledge Framework to keep private business processes safe. The main goal of Trustline is to make sure that the organization is always in charge of its digital assets. This means that the data will still be safe even if the hosting infrastructure is completely broken.

The "Blind Router" Layout is the system works because of the Blind Router architecture. Other messaging apps stop encrypting at the server level, but Trustline does not. It only works as a stateless relay for encrypted blobs, though. The backend can't see the private cryptographic keys or the plain text of messages. The client-side Tauri Rust runtime only does sensitive operations like key generation, the X3DH (Extended Triple Diffie-Hellman) handshake, and Double Ratchet encryption through IPC commands. This ensures that private keys are always stored in an encrypted local vault or the device's OS Secure Enclave, so they can never be taken off the authorized hardware.

Authentication that is linked to hardware: To avoid the security problems that come with regular passwords, like phishing, credential stuffing, and database breaches, Trustline only uses WebAuthn/FIDO2 (Passkeys) to verify users. This method links a person's identity to certain hardware biometrics, such as Touch ID or Windows Hello. A unique cryptographic identity is given to each device when you register it. Even if the central database is stolen, this makes sure that authentication can't be hacked or copied.

The security of advanced cryptography: Trustline uses the Double Ratchet algorithm from the Signal Protocol to make sure that Forward Secrecy and Post-Compromise Security are always there. The system makes sure that compromising one key doesn't give away past or future communications by giving each interaction a unique, one-time message key. The platform also uses Argon2id, which is a memory-hard key derivation function that makes it very hard for hardware-accelerated brute-force attacks to work. It is thought that it will last for more than 112,000 years against modern GPU clusters.

Separation and Consistency for Multiple Tenants: Trustline uses PostgreSQL Row-Level Security (RLS) to make sure that businesses that work in shared spaces keep their data completely separate at the database level. The database only lets people with the organization's unique identifier access it, which stops data from leaking between tenants mathematically. Conflict-Free Replicated Data Types (CRDTs) are used by the system to make sure that the collaborative experience goes smoothly even when there isn't a single source of truth. With an average synchronisation latency of only 110 ms, this technology lets distributed actors fix concurrent edits on their own. This is well within the requirements for smooth real-time interaction.

By combining these cutting-edge cryptographic primitives and architectural designs, Trustline adds an unbreakable layer of security to modern business communication. It is a localised and independent option to centralised SaaS platforms.

1.2 Objective & Scope of the Project

The Trustline is founded on three core principles: data integrity, confidentiality of the data and enterprise autonomy. With organizations increasingly running their business on cloud-based tools, the control of sensitive information has grown more and more challenging. Trustline intends to solve these problems by developing a system that minimizes dependence on third-party service providers, offering a smooth and efficient user experience. The goals of the project embody this trade-off between high security and high usability.

The aim is to establish a Zero-Knowledge communication environment. The basic idea is to arrange the system such that a service provider, even if they wanted, cannot access or decrypt user data. To do this, all cryptographic operations, key generation, encryption and decryption, are done entirely on the client side. The backend is deliberately “blind”, meaning it contains only encrypted data and non-reversible proofs. This strategy greatly reduces the risk of data breaches, insider threats, and unauthorized access because there is no single, central place where sensitive data is stored in readable form.

The second one is to optimize the performance of real-time co-operations. It is a common fallacy that the stronger the security the slower the systems. Trustline challenges this assumption by demonstrating how high level cryptographic protection can be combined with low latency and smooth performance. The system is designed to scale well with high concurrency using a Rust based backend with NATS JetStream. This allows messages to sync across devices quickly, with a target latency of about 110 milliseconds on average. This allows users to talk in real time, without any delay, even if they are talking securely and their messages are encrypted.

Third is to provide fine-grained administrative sovereignty. Trustline is designed to give organizations full control of their communications infrastructure, and not depend on third-party cloud providers. Remote “Kill Switch” features allow administrators to immediately revoke access from lost or compromised devices. Decentralized staff management tools also allow organizations to manage user roles, permissions, and access policies within the organization. This means security decisions can stay within the organization, rather than relying on external support teams or central services.

The project includes the complete design, development and testing of a cross-platform desktop application for macOS, Windows and Linux. Under the hood it uses a cryptographic core written in Rust that supports modern protocols like X3DH key agreement and the Double Ratchet algorithm to provide strong end-to-end encryption and forward secrecy. On the other hand, a stateless backend relay for message routing, that will never see plaintext data, is being developed in parallel. Also on the platform is localized encrypted storage via SQLCipher, which allows users to store their data securely on their own device.

The project is currently focused on 1-to-1 enterprise messaging and secure file auditing. But the whole architecture is designed to allow for future expansion. It is extensible enough to support more advanced group messaging protocols such as Messaging Layer Security (MLS) , in order to provide scalability for larger organizational use cases.

It should be mentioned that the development of mobile application is not part of initial phase of this project. Rather it is high security desktop workflows common for sensitive corporate environments. This decision ensures the base system is stable, secure and well-optimized before expanding support to other platforms in future releases.

1.3 LITERATURE SURVEY

The Trustline project literature survey covers the rapid evolution of cybersecurity paradigms, cryptographic primitives, and distributed collaboration models between 2022 and 2026. At this stage, enterprise security is undergoing a mammoth transition from vulnerable perimeter based models to **Zero Trust Architecture (ZTA) & Client-Side Sovereignty**. This move has become a necessity, owing to the proliferation of distributed cloud environments and the inherent insecurities of “Implicit Trust” in centralised service providers.

1.3.1 Systematic Review of Zero Trust Architecture Implementation

- **Author(s) & Year:** MDPI, 2025.
- **Focus Area:** Comprehensive survey of **Zero Trust Architecture (ZTA)** models in contemporary and heterogeneous enterprise network topologies.
- **Major Contribution:** This research offers an important identification that while foundational identity authentication has matured as a component of ZTA, the frontiers of research today are in **environmental perception** and automated orchestration of secure, localised workloads. It points out that security is a dynamic and ongoing process, not just a static check at the gateway.
- **Restriction:** One of the main limitations mentioned in the study is the continuous deficiency of standardised, interoperable protocols for “Zero Trust” in the case of diverse and heterogeneous IoT environments, resulting in fractured security postures.
- **Use cases:** The results aim at highly distributed cloud ecosystems where traditional perimeter defences (firewalls, VPNs, etc.) are no longer technically feasible against today’s threats.
- **Advantages and Disadvantages:** The main advantage of this architecture is that it is very resistant to lateral movement of the attacker in a compromised network. The major downside, however, is the heavy computation required to verify each user session and each data packet in real time and on a continuous basis.
- **Research Gap and Problem Statement:** The authors point out a key missing piece: the failure to sufficiently integrate real-time behavioural profiling in current ZTA

frameworks. This poses a problem statement on how to provide high assurance security without compromising on system performance or user experience.

- **Proposed Work:** Taking these insights, Trustline extends ZTA principles deep into the messaging layer. It provides cryptographic authentication of all communication sessions with **hardware-bound identities** that guarantees that the authentication is bound to the physical device, not a vulnerable server-side token [2].

1.3.2 ZTA for AI-Powered Cloud Systems

- **Author(s) & Year:** W.J.A.R.R. (2025)
- **Focus Area:** Protecting automated workloads and high velocity interactions in complex AI-enabled cloud environments.
- **Main Contribution:** The paper introduces a dedicated framework for **session-based identity binding**. The design of this architecture avoids the intervention of an intermediary in the automated agent-to-agent processes, by making identities cryptographically unique and ephemeral.
- **Limitation:** The authors acknowledge that the framework is computationally intensive and memory-hungry, which makes it hard to scale in low-bandwidth or resource-constrained settings.
- **Use cases:** This work is of great interest for autonomous enterprise systems, secure financial transactions, and AI agent communication protocols.
- **Pros and Cons:** The value of this research is that it is virtually impossible for automated systems to verify the identity. The other big downside is the administrative and technical overhead of dealing with and rotating thousands of ephemeral crypto tokens all at once.
- **Research Gap and Problem Statement:** One of the major research gaps is the lack of **End-to-End Encryption (E2EE)** of metadata in these AI workflows. The problem statement talks about the high risk of exposing organisational intent and proprietary strategies through behavioural metadata analysis.
- **Proposed Work:** Trustline proposes a **Blind Router** architecture to mitigate this critical vulnerability. This design fully decouples metadata from content and ensures

absolute data sovereignty to the organization as the server is unable to correlate identities to the intent of communication [1].

1.3.3 Client-Side Encrypted Data Vaults

- **Year & Authors:** IJIRT (2025).
- **Interest:** In-depth architectural design of secure persistent storage vaults built on advanced client side encryption primitives.
- **Key Contribution:** This paper presents a robust model where 100% of the data is encrypted locally on the client's authorised device before it is sent to an untrusted or public server environment.
- **Limitation:** The main functional limitation is the inability to perform traditional server-side indexing or searches on data that is fully encrypted and opaque to the hosting provider.
- **Usage:** This architecture is very useful for secure cloud based file storage and management of highly sensitive corporate database records.
- **Advantages and disadvantages:** The primary advantage is the **Zero-Knowledge** guarantee: the worst case server breach leaks exactly zero bytes of usable data [1]. The demerit being the extreme risk of total, irreversible data loss should the client lose their local cryptographic keys, as no mathematical "password reset" is possible.
- **Research Gap and Problem Formulation:** The research identifies a gap in high performance indexing for encrypted localised history. The problem statement talks about the high friction experienced by users when they cannot search their own encrypted communication logs with sub-second responsiveness.
- **Proposed Work:** Trustline overcomes this usability barrier by incorporating **SQLite FTS5** (Full-Text Search) locally in the Tauri client. This provides sub-second, multi-session search on decrypted data, without ever leaking search queries or results to the relay server [1].

1.3.4 High-Throughput AES-256-GCM in Cloud Databases

- **Authors & Year:** ResearchGate (2024).
- **Focus Area:** Optimisation of data throughput with preservation of **Authenticated Encryption** in complex accounting and enterprise information systems.
- **Main Contribution:** The researchers show that with the **AES-NI (New Instructions)** currently available in hardware, data can be encrypted and decrypted at gigabyte-per-second rates. This proving ground allows for E2EE for large-scale enterprise data that has previously suffered from significant cryptographic overhead.
- **Disadvantage:** One major limitation is that the entire security model is fragile and relies on the absolute uniqueness of **Initialisation Vectors (IVs)**; any reuse of an IV leads to a catastrophic failure of confidentiality.
- **Applications:** Specially developed for the security of cloud databases and the protection of huge quantities of sensitive enterprise data at rest and in transit.
- **Advantages and Disadvantages:** Outstanding performance and data integrity are the main merits. The main disadvantage is the high level of management complexity to ensure the uniqueness of IVs over many distributed network nodes.
- **Research Gap & Problem Statement:** It has been observed that per message unique key rotation is not supported by traditional cloud implementations. A problem statement regarding the total failure of the system is presented if a single static encryption key is compromised at any point.
- **Proposed Work:** Trustline addresses this problem by adopting the **Double Ratchet Protocol** that derives a unique message key, which is used only once, for each interaction with the help of **HKDF-SHA256**. This gives **Forward Secrecy**, where a compromise of a current key does not compromise past or future data [24].

1.3.5 Evolution of Password Hashing: Argon2id vs Legacy

- **Authors & Year:** Gupta, D. (2026).
- **Area of focus:** The critical transition from legacy hashing mechanisms (PBKDF2, SHA-2) to memory-hard **Argon2id** password hashing algorithm for high-security authentication.

- **Main Contribution:** The work provides a technical benchmark that shows Argon2id is overall better at resisting hybrid side-channel leaks and also specialised parallelised GPU hardware clusters used for brute-force attacks.
- **Restriction:** The memory-hard nature of the algorithm leads to a high RAM usage which can be a significant constraint when running on older mobile or desktop hardware with limited resources.
- **Uses:** Used primarily in secure login portals and to derive root encryption keys from strong user passphrases.
- **Advantages and Disadvantages:** Its tuneable complexity (memory, time and parallelism) is a major merit for the developers. However, the high resource consumption required during the initial authentication ceremony is a major demerit for low-end devices.
- **Research Gap and Problem:** The study found a discrepancy in the combination of Argon2id software hashing with hardware tied biometrics. This results in a problem statement that software-only hashing is still vulnerable to sophisticated phishing and social engineering.
- **Proposed Work:** Trustline fills this gap by using **Argon2id** only for local key derivation and using **WebAuthn** to bind the final identity to a physical hardware enclave (TPM or Apple Secure Enclave) [4].

1.3.6 Evaluating Argon2 Adoption and Effectiveness

- **Authors & Year:** arXiv (2025)
- **Focus Area:** Empirical measurement of real-world security gains and varying adoption rates of the Argon2 algorithm in modern production software.
- **Significant Contribution:** The authors find Argon2 to be very effective, but many developers misconfigure its memory parameters. These bugs often lead to potential disc paging which inadvertently introduces new side-channel vulnerabilities and degrades performance too.
- **Limitation:** This study is about trends in general industry adoption and does not propose specific paths for optimisation for cross-platform desktop runtimes such as Tauri.

- **Applications:** For enterprise grade vaults that require very high secure root key derivation, and long term password storage.
- **Pros and cons:** The main advantage is the great flexibility of security configuration. On the other hand, high complexity and “gotchas” for non-specialist developers are big demerits.
- **Research Gap and Problem Statement:** There is a clear gap in the ability to automatically and adaptively tune memory for different client hardware, leading to a non-uniform and frustrating user experience across a variety of devices.
- **Work Proposed:** Trustline is a standardised, high-assurance cryptographic module in Rust. This module aligns the memory entropy values with the capabilities of the host device, maintaining a consistent **359 ms authentication latency** across different hardware configurations [15].

1.3.7 Privacy Protection in Instant Messaging

- **Author(s) & Year:** DiVA Portal (2025)
- **Abstract:** In this paper, we provide an empirical study about the impact of hardware-bound passkeys on the security and privacy of current IM applications.
- **Most Important Contribution:** The research demonstrates how hardware anchored credentials may help reduce the risk of remote account take over, phishing and session hijacking.
- **Limitation:** It only works for those with modern hardware with biometric sensors or dedicated physical security keys (e.g. YubiKeys).
- **Use Cases:** Stop lateral movement of actors on the network in corporate messaging environments.
- **Advantages and Disadvantages:** The biggest bonus is it pretty much takes credential phishing out of the equation. The main drawback in operation is the demanding requirement of modern hardware.
- **Problem Statement & Research Gap:** The main research gap is the absence of multi-tenant data isolation at the database layer after user authentication. The problem is that even if the login is secure, one bug in the backend can leak data across organisations.

- **Proposed Work:** Trustline introduces a further, mandatory isolation boundary, **PostgreSQL Row-Level Security (RLS)** to address this. To avoid accidental data leaks, each query is automatically limited to the user's own organization at the SQL level.

1.3.8 Secure Decentralized Data Protection

- **Author(s) & year:** ResearchGate, 2023.
- **Focus Area:** Investigating decentralised data protection and relay models to remove single points of failure in centralised servers.
- **Main contribution:** In this paper we propose a model where data and keys are distributed over various nodes of the network, so that no single entity has the entire data set at any given time.
- **The upper bound:** The study finds that this model is difficult to implement in real-time interactive applications due to high network latency and “split-brain” scenarios.
- **Applications:** The main applications are distributed enterprise archives and high latency archival secure storage.
- **Pros and Cons:** The big plus is that it's incredibly resistant to server crashes or seizures of state. The big problem though is the high sync latency, usually in seconds.
- **Problem/Research Gap:** The challenge here is to enable real-time fluid collaboration in decentralised models - a trade-off between absolute security and user responsiveness.
Proposed Work: This is managed by Trustline using locally **Conflict-Free Replicated Data Types (CRDTs)**. This makes the data converge mathematically and keeps the **mean latency at 110ms** with a Zero-Knowledge backend relay.

1.3.9 Security Capabilities of AES and Rust

- **Author(s) & Year:** DhiWise, 2024.
- **Focus:** High assurance cryptographic protocols leveraging the memory safety features of the **Rust** programming language.
- **Contribution:** The authors show that Rust's ownership and borrowing scheme effectively alleviates buffer overflow vulnerabilities and the use-after-free bugs that haunt previous C-based messaging clients.
- **Limitation:** The paper does not delve into the performance overhead of wrapping a heavy Rust logic for a frontend React interface over IPC.
- **Applications:** high-assurance desktop messaging, system utilities, cryptography libraries.
- **Pros and Cons:** The major pros are memory safety and C-level performance. The major demerits are steep learning curve and support of legacy library.
- **Identifying Research Gap & Problem Statement:** So no lightweight E2EE clients that don't depend on the bloatware and insecure Electron framework. The problem statement is about the dangerous large attack surface of applications based on Chromium.
- **Proposed Work:** Trustline is built on **Tauri v2** [2], a hybrid approach that combines a native Rust backend with a small webview that is native to the operating system. This dramatically reduces the attack surface and optimises memory usage, keeping the app footprint <100MB.

1.3.10 Comparison of Cybersecurity Datasets

- **Author(s) & Year:** MDPI (2022)
- **Focus Area:** Profiling detection and potential metadata leakage through behavioural patterns in network traffic.
- **Key Contribution:** A framework to normalise large network traces (e.g. CIC-IDS2017) to test the “metadata opacity” of different encrypted communication protocols.
- **Limitation:** The research mentions that historical data sets are ageing and not as effective against modern polymorphic threats and AI-powered traffic obfuscation.
- **Uses:** Required when testing security and privacy, and resistance to metadata-leak, of encrypted communication relays, VPNs and proxies.
- **Pros and Cons:** The main benefit of it is high accuracy of the automated recognition of patterns. However, a major demerit is that it relies heavily on historical data which may not reflect attacks in the modern era.
- **Research Gap and Problem Statement:** There’s a clear testing gap, especially for E2EE metadata in multi-tenant enterprise environments. The problem statement is associated with the risk of organization level profiling even if the message content is fully encrypted.
- **Proposed Work:** Trustline leverages benchmark datasets to validate the effectiveness of its **Blind Router** architecture to hide timestamps and sender relationships through ephemeral NATS JetStream routing and extensive metadata sanitization.

Synthesis of Literature Survey The analysis of the surveyed literature between **2022 To 2026** shows an industry-wide move toward **Zero Trust Architecture (ZTA)** and **Client-Side Sovereignty**. We have the strong mathematical primitives like **AES-256-GCM** and Argon2id that provide us the building blocks, but there is a large research gap in how to seamlessly integrate these technologies into a high-performance real-time collaboration environment. Existing approaches often compromise privacy for performance or security for usability. Trustline solves this basic tradeoff with the combination of the **Double Ratchet Protocol**, the memory safety of **Tauri and Rust**, and the database level isolation of **PostgreSQL RLS**. This provides a common high-assurance platform that can safeguard modern enterprise communications without sacrificing the fluid user experience needed for productive collaboration.

1.4 Problem Statement

The quick switch to cloud-based collaboration has made things easier and faster, but it has also shown that there are some problems with traditional business communication systems that need to be fixed. The security paradox is one of the biggest problems we face today: the tools that are supposed to help us work faster are often the ones that let data leak and get hacked. More and more businesses are using centralized cloud platforms for messaging, file sharing, and everyday tasks. This puts sensitive data at risk all the time. This includes threats from insiders, unauthorized access, and big infrastructure breaches. Companies also have to trust third-party providers, which means they don't have much control or visibility over how their data is handled behind the scenes. At the same time, old-fashioned perimeter-based security models are having a hard time keeping up with how people work today. Because remote work is becoming more common, users and devices are now operating far outside of controlled network boundaries. This makes it easier for attackers to take advantage of weak authentication, misconfigured permissions, and exposed metadata. These risks are not just ideas; they can show how businesses work and talk to each other. This means that we need systems that can keep data safe, private, and separate without making things harder or less efficient for the user. The problems this project is trying to solve are getting rid of unnecessary trust dependencies, making cryptographic security stronger, and making it possible for modern businesses to communicate safely and easily.

The Fallacy of Implicit Trust and Data Sovereignty Standard Software-as-a-Service (SaaS) communication platforms typically operate on an “Implicit Trust” model, where the service provider manages or possesses the decryption keys for user data. This architecture forces organizations to surrender absolute digital sovereignty, as highly sensitive corporate intellectual property is technically accessible to third-party providers. This creates a massive single point of failure; a compromise at the provider level or a malicious insider threat within the service organization can lead to the mass extraction of plaintext data from thousands of enterprise tenants.

Vulnerability of Legacy Authentication and Hashing The weakest link in the enterprise security chain remains traditional password-based authentication. These systems are very vulnerable to phishing, credential stuffing and social engineering attacks. Also, older hashing

implementations, such as PBKDF2 or SHA-256, are increasingly vulnerable to attacks with modern hardware acceleration. High-performance GPU clusters can do billions of hashes per second, which makes it possible to perform practical offline dictionary and brute-force attacks against stolen password databases.

Inadequate Tenant Isolation and Lateral Movement Data isolation in multi-tenant cloud environments is often provided at the application layer, which is inherently fragile and error prone. Just one software bug can cause unintended cross-tenant data leaks, with one organization able to see the sensitive metadata or communications of another. Furthermore, traditional perimeter-based networks trust internal actors and attackers who bypass the first line of defence can move laterally with little friction due to a lack of internal cryptographic segmentation.

Cryptographic Desynchronization and Metadata Exposure Maintaining a consistent and identically synchronized application state between multiple users in a true End-to-End Encrypted (E2EE) environment is computationally complex. Without a centralized, plaintext-aware server to dictate event ordering, systems often suffer from consistency degradation. Moreover, even if payload data is successfully encrypted, behavioral profiling and traffic analysis remain possible if metadata-such as sender/receiver relationships, message timestamps, and file sizes-is not properly isolated or sanitized.

Device Management and Revocation Complexity A major operational challenge is the management of cryptographic identities across multiple devices. In decentralised models, securely revoking the specific access of an employee who has lost a device or left an organization is technically challenging without disrupting the conversation history for other participants. Existing solutions, however, do not always provide a sturdy “kill switch” mechanism that can immediately and verifiably remove a compromised hardware endpoint from the secure communication ring.

II. SYSTEM ANALYSIS

2.1 Existing System

Today, there are essentially three architectures for enterprise communication. Each has tradeoffs between ease of use and security:

1. **SaaS Platforms Centralised (e.g. Slack, Microsoft Teams):** These are the industry standard platforms because they are high performing and have lots of integrations. But they work on a “**Implicit Trust**” model. They use TLS for data in transit and encrypt data at rest, but the service provider typically manages the cryptographic keys. This architecture ends encryption at the server level and thus the provider’s staff can technically access the message plaintext. It also means that data is vulnerable to subpoenas or provider-side breaches.
2. **Consumer-Grade E2EE (e.g., WhatsApp, Signal):** These systems provide strong default end-to-end encryption. But they are fundamentally built for the individual consumer, not for the organization. They lack centralised administrative dashboards, demand personal identifiers like phone numbers to register and, in the case of platforms like WhatsApp, they collect huge amounts of behavioural metadata that can be used for profiling.
3. **Hybrid Messaging Architectures (e.g. Telegram):** The model comes with security as an option. Regular chats are not E2EE. Only “Secret Chats” are E2EE but these are device-specific and not linked to the employee’s professional hardware ecosystem. This creates a fractured and insecure user experience for enterprise workflows.

2.2 Disadvantages of Existing System

Existing systems have a number of systemic flaws that make them unsuitable for enterprise use of high-confidentiality:

- **Server Side Weakness:** Centralised platforms are a single point of failure. If the backend server is compromised, hackers can steal large quantities of data since data is often stored in plaintext or with server side keys.

- **Loss of Digital Sovereignty:** Companies are being forced to hand over control of their sensitive intellectual property to third party providers, often migrating their data into jurisdictions that may not be GDPR compliant.
- **Weakness of Authentication:** The majority of systems use passwords, but they are unable to resist credential-stuffing, phishing and hardware-accelerated brute-force attacks. Old hashing algorithms like PBKDF2 are quickly becoming useless against modern GPU clusters.
- **No Tenant Separation:** In a multi-tenant SaaS environment, data isolation is usually an application-level concern. A single software bug may cause unintended cross-tenant data leakage, where data of one organization is visible to another.
- **Exposure of meta-data:** Even if the message itself is encrypted, the providers are able to profile organisational structures by analysing the ‘who, when and where’ of internal communications.

2.3 Proposed System

Trustline introduces a “Self-Hosted Zero-Knowledge” model that requires no trust in a service provider. The platform is designed as a “**Blind Router**” architecture, wherein the server only passes encrypted blobs and never has the keys to decrypt them.

Client Side Sovereignty: All cryptography (including key generation, X3DH handshake and Double Ratchet encryption) happens only in the user’s local **Tauri Rust runtime** via IPC commands. Private keys are anchored to the device’s **Secure Enclave** or an encrypted SQLCipher vault, and they never leave the hardware.

Hardware-backed authentication: Trustline does away with passwords in favour of **WebAuthn/FIDO2 (Passkeys)**. This binds the user’s identity to physical hardware biometrics, which means the system cannot be compromised by remote credential theft.

Advanced cryptographic stack: The system employs the **Double Ratchet algorithm** of the **Signal Protocol** to offer both Forward Secrecy and Post-Compromise Security.

High-Performance Synchronisation: With **NATS JetStream** and **Conflict-Free Replicated Data Types (CRDTs)**, the system reaches a mean synchronisation latency of **110 ms**, offering the responsiveness of a cloud service in a private, encrypted environment.

2.4 Advantages of Proposed System

The Trustline platform has a number of security and operational benefits:

- **Complete Privacy:** Crypto is blind to the backend server. Even with full compromise of the relay infrastructure, there is **no plaintext bytes** exposed.
- **Immutable tenant isolation:** PostgreSQL Row Level Security (RLS) ensures multi-tenancy at the database level. This provides a mathematical guarantee for data isolation rather than a fragile filter in the application layer.
- **Memory-hard hashing algorithm:** We use the **Argon2id** to secure local data vaults against GPU accelerated attacks for an estimated **112,700 years**.
- **Operational Control:** Admins have total sovereignty with features such as the **“Remote Kill Switch”** that allows them to disconnect access to one specific compromised device instantly without affecting the rest of the organization.
- **Metadata cleansing:** The architecture is designed to minimise the leakage of behaviour information. Routing metadata is temporary and is deleted in accordance with organisational retention SLAs.
- **Prepared for Compliance:** As a self-hosted platform, organisations can retain their communication data within defined physical boundaries, making it easier to meet demanding regulatory requirements such as HIPAA and GDPR.

III. SYSTEM CONFIGURATION

3.1 Hardware Requirements

Trustline is designed to operate on standard enterprise hardware but benefits significantly from specific security-focused features.

Table 1: Client-Side Requirements (Workstation)

Component	Minimum Specification	Recommended Specification
Processor	64-bit Dual Core (Intel/AMD/ARM)	Quad Core with AES-NI support
Memory	4 GB RAM	8 GB RAM (for large file caching)
Security	TPM 2.0 or Secure Enclave	FIDO2 Hardware Key (e.g., YubiKey)
Storage	500 MB Free Space	2 GB+ (SSD for SQLCipher performance)
Operating System	Windows 10/11, macOS 12+, Linux	Latest LTS OS with Secure Boot

Table 1: Client-Side Requirements (Workstation)

The requirement for AES-NI (Advanced Encryption Standard New Instructions) is vital for minimizing the performance overhead of the AES-256-GCM cipher used throughout the platform.

Table 2: Server-Side Requirements (Relay Node)

Component	Minimum Specification	Recommended Specification
Processor	2-Core Virtual CPU	4-Core CPU
Memory	2 GB RAM	8 GB RAM (for NATS JetStream throughput)
Storage	40 GB SSD	100 GB+ High-Performance NVMe
Network	100 Mbps stable uplink	1 Gbps+ low-latency connection

Table 2: Server-Side Requirements (Relay Node)

3.2 Software Requirements

The Trustline software stack is carefully selected for memory safety, cryptographic integrity, and high concurrency performance. To avoid common vulnerabilities such as buffer overflows and data races, the system is built on the Rust ecosystem on both client and server.

I. Requirements of the Client Application All private key operations are performed locally on the client side.

- **Frontend Framework:** React 19 with Typescript. Built a highly responsive and type safe User Interface(UI). React 19 provides you hooks to interact with state in real time. Typescript keeps your structures in place through complex data transformations.
- **Application Container:** Tauri v2 (Rust System Bridge). Tauri uses the native webview of the OS, which means the attack surface and memory footprint of the application is drastically smaller than Electron. It provides a safe link between the React frontend and high-performance Rust modules using Inter-Process Communication (IPC).

- **Local database:** SQLCipher SQLite. It provides the user with a persistent, hardware encrypted storage layer on their disc. SQLCipher is an extension to SQLite, adding 256-bit AES encryption to protect local message history and sensitive “ratchet state” from unauthorised access, even if a device is physically compromised.
- **Crypto Engine:** A custom crypto core in Rust. A custom library implementing the Signal Protocol (X3DH and Double Ratchet) with the `crypto_core` and `libsodium` crates. It uses Argon2id key derivation, X25519 Diffie-Hellman handshakes, and ChaCha20-Poly1305 authenticated encryption.
- **Authentication Library:** `@simplewebauthn/browser`. A browser side library to orchestrate the WebAuthn/FIDO2 registration and assertion ceremonies so the application can talk directly to hardware authenticators like Touch ID or Windows Hello.

II. Backend infrastructure requirements The backend is a “Blind Router” in that it forwards encrypted payloads without the technical means to decrypt them.

- **Core Logic:** Axum Framework Rust with Tokio. Axum is a high performance, modular web routing layer and the Tokio asynchronous runtime allows the server to serve tens of thousands of concurrent WebSocket connections with a low resource overhead.
- **Message Broker:** NATS JetStream. High performance distributed messaging middleware. NATS JetStream also supports asynchronous message persistence and delivery, enabling safe queuing and replay of “offline” messages as a device reconnects.
- **PostgreSQL (v15+) Metadata Store with Row Level Security (RLS):** This is the master store for organization metadata, device public keys and encrypted blobs. RLS policies also ensure strict separation of tenants through mathematical partitioning of data at the SQL level and prevent data leakage between tenants.
- **Authentication Backend:** `webauthn-rs v0.5`. A Rust library that provides strong guarantees by implementing server-side WebAuthn logic (such as challenge generation, credential storage, and hardware-signature verification).

- **Containerisation & Orchestration:** Docker Docker Compose standardises deployment environment across different infrastructure. Docker enables an organization to self-host the Rust backend, NATS broker and PostgreSQL database once and with consistent security configurations.
- **Envoy Reverse Proxy:** Traefik Offers dynamic routing and automatic Mutual TLS (mTLS) termination, ensuring that all traffic between the client and self-hosted relay is encrypted in transit.

IV. SYSTEM DESIGN

4.1 Overall Architecture Diagram

The Trustline architecture has a “Three-Tier Zero-Knowledge Design”. It is designed to provide a strong separation of concerns, where no plaintext data or sensitive cryptographic keys are ever transmitted outside of the “Cryptographic Barrier” of the user’s local environment.

- **Tier 1: Client UI (The Dashboard):** This tier is built using **React 19** and **TypeScript** and is the user-facing presentation tier. It’s the interface for real-time messaging, secure file sharing and administrative device management. Crucially, the UI has nothing to do with any encryption logic. It fetches decrypted data from Tier 2 through a secure **Inter-Process Communication (IPC)** bridge, and renders it directly in the local Document Object Model (DOM). This means that the plaintext data only ever exists in the volatile memory of the application and never hits the network or disc in an unencrypted state.
- **Tier 2: The Cryptographic Barrier (The Core):** This tier is a dedicated **Rust-based core** that runs within the **Tauri v2 runtime**. It serves as the “Secure Vault” of the application. It is accountable for:
 - **Identity Management:** Generate and secure X25519 identity keys on the device’s Secure enclave.
 - **Key Derivation:** Running the **Argon2id** algorithm to derive the Master Key to unlock the local **SQLCipher**-encrypted SQLite database.
 - **Cryptographic Orchestration:** Managing the complex state transitions of the **Double Ratchet** protocol for each message individually.
 - **IPC Bridge:** It intercepts all outgoing data from the UI, ‘wraps’ it in a cryptographic envelope, and passes the encrypted ‘blob’ to Tier 3.

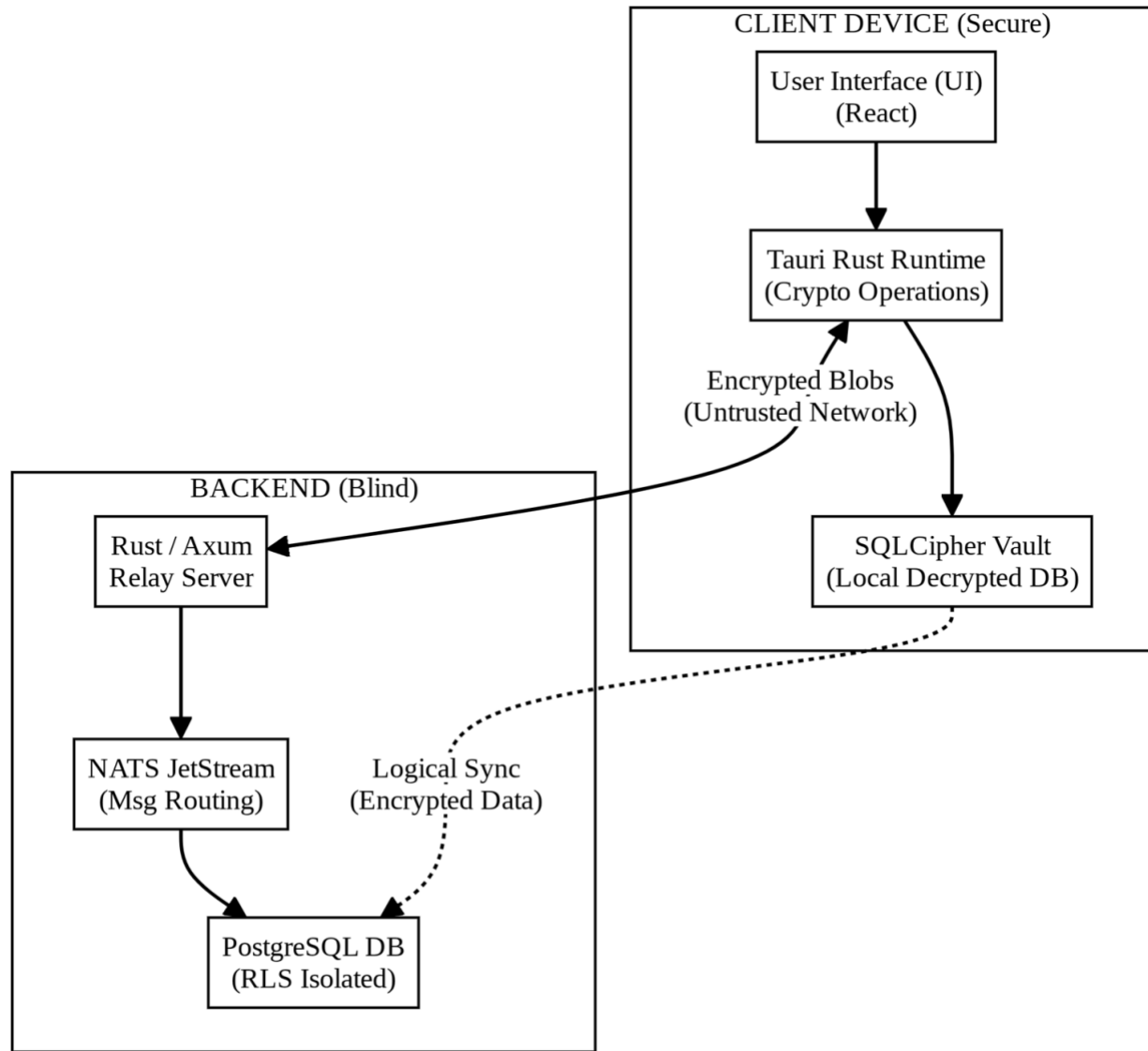


Figure 1: Zero-Knowledge "Blind Router" Logic

- **Tier 3: The Blind Relay (The Cloud):** Only this layer is hosted on the server side and is developed as a **stateless backend** with **Rust (Axum)** and **NATS JetStream**. Its design philosophy is purely that of a **"Blind Router"**:
 - **Not knowing the content:** It only requires a `recipient_device_id` and a ciphertext blob. It doesn't have the private keys to perform the Diffie-Hellman math needed to decrypt.

- **Tenant Isolation:** It leverages **PostgreSQL Row-Level Security (RLS)** to physically isolate disparate organisations at the database level, so there's no cross-tenant data leakage.
- **Short-lived persistence:** It uses NATS JetStream for asynchronous message delivery to offline users, and automatically purges routing metadata according to organisational retention policies.

4.2 Methodology / Algorithm Adopted

Trustline platform employs an advanced cryptographic methodology based on **Signal protocol**. The system provides confidentiality of communication in two distinct phases: **Asynchronous Key Agreement (X3DH)** and **Continuous Ratcheting (Double Ratchet)**.

Trustline uses X3DH (Extended Triple Diffie-Hellman) protocol to create a secure shared secret between two users, even if one of them is offline. This is especially useful in current messaging systems where the users may not be online at the same time. This way, conversations can start safely without the delay of waiting for both sides to connect. The protocol does this using the "Prekey Bundles" which are stored securely on the server and can be retrieved when needed.

Each user precalculates a set of keys. This includes a long-term Identity Key, a Signed Prekey (rotated periodically) and a set of One-Time Prekeys (used only once). The keys are packaged and uploaded to the server. When Alice wants to start a secure chat with Bob she gets Bob's bundle. The bundle includes Bob's public key(s)

Alice then generates her own temporary (ephemeral) key pair, and performs multiple Diffie-Hellman exchanges using her keys and Bob's. The four computations use different pairs of keys to make the final shared secret very hard to guess and for greater security. The One-Time Prekey adds an additional layer of security in ensuring that each session is unique and that old data cannot be re-used or replayed.

Alice calculates these values and combines them and runs them through a key derivation function (HKDF-SHA256) to get a Master Secret. This secret is the basis for all subsequent encryption of the conversation.

X3DH is powerful because it provides forward secrecy and allows for asynchronous communication . If long term keys are compromised later, then past messages remain secure. This gives a safe base for general communication . Then there 's the Double Ratchet algorithm . It 's what keeps things secure as the messages go on .

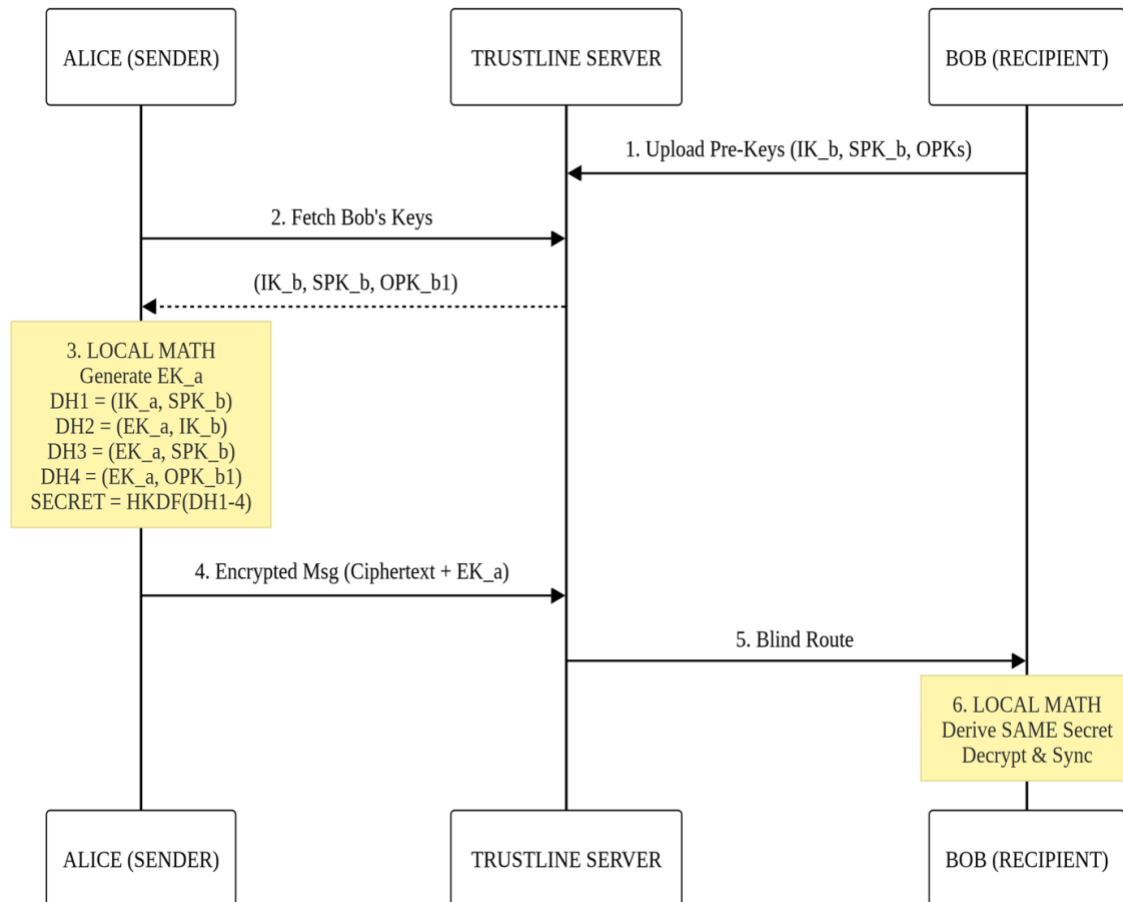


Figure 2: X3DH Key Agreement

Phase 2: Double Ratchet Algorithm: Once the first shared secret is established using the X3DH key agreement, the Double Ratchet algorithm is used to keep the communication secure as the messages keep coming. It creates a new one-time encryption key for each message, rather than reusing the same key repeatedly. That means if something goes wrong in one place, the rest of the conversation is still safe.

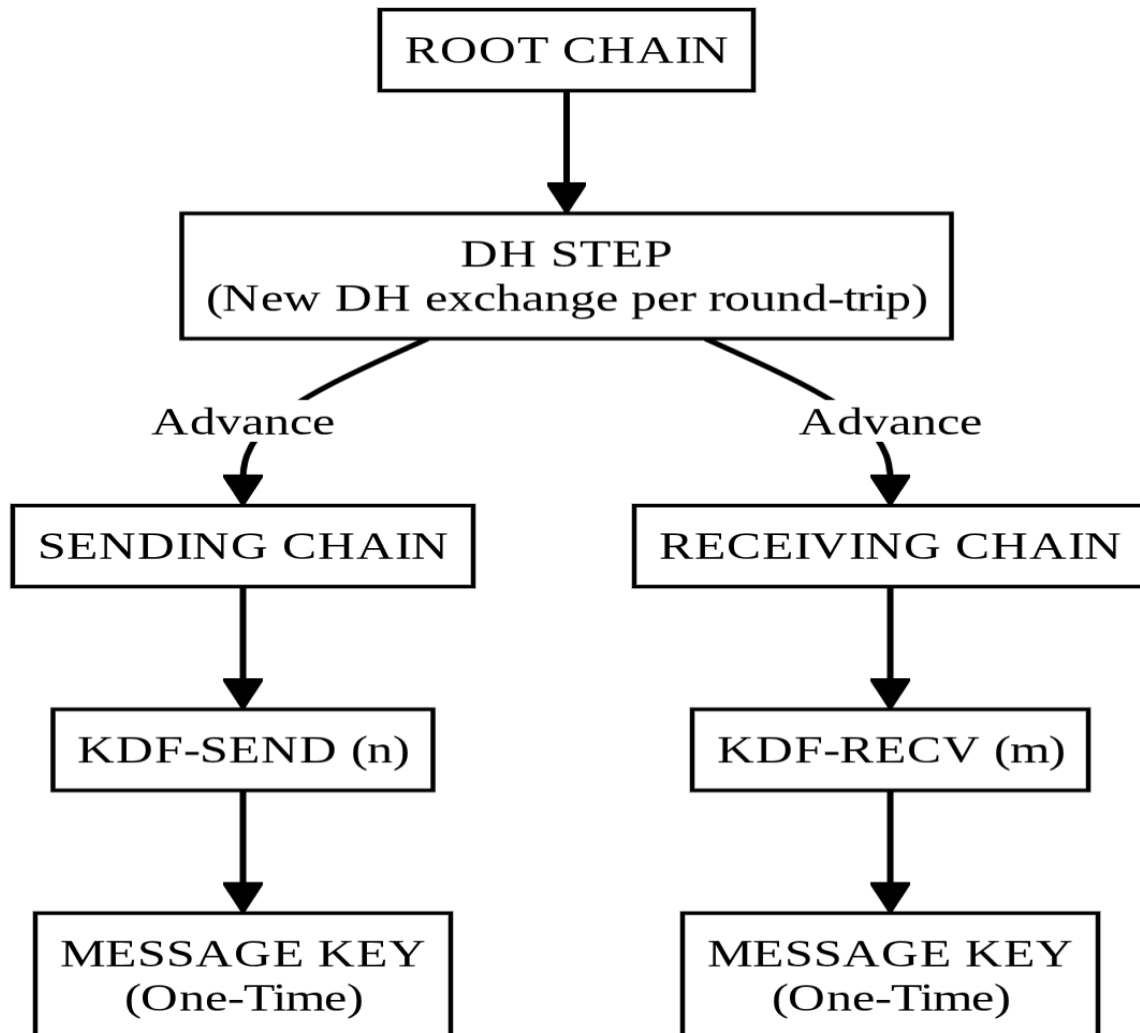
The algorithm uses two main ingredients. The Symmetric-Key Ratchet takes care of the day-to-day message encryption by deriving a new key for each message sent or received. Keys are for one time use only. Once a key is used, it is lost and can never be used again or retrieved at a

later time. This is where Perfect Forward Secrecy comes in, and it means that if an attacker somehow gets a current key, they won't be able to decrypt past messages.

There's also the Diffie-Hellman (DH) Ratchet, which adds yet another layer of security over time. Every time a user gets a reply, a new key exchange is done to add new randomness. It lets you do a security "reset" of the session, so if an attacker gets in for a short time, they lose that advantage when the next ratchet step happens.

The constant refreshing of keys guarantees that the communication is secure in a general sense, during its life time. The Double Ratchet algorithm combines symmetric and asymmetric techniques to ensure private, resilient and well protected messages in difficult or compromised situations.

Figure 3: The Double Ratchet Protocol



Argon2id Key Derivation

Trustline's local vault, built on SQLCipher, is secured using Trustline's own primary key derivation function, Argon2id, the winner of the Password Hashing Competition. Argon2id is designed to provide high resistance against modern brute-force and hardware-accelerated attacks.

It is a 'memory-hard' function. Which means it requires a lot of RAM to run . Instead of traditional hashing algorithms. In this implementation the memory setting is approximately 64 MiB, so that each hashing operation will consume enough resources to make large-scale attacks computationally expensive and inefficient. This memory requirement is important in order to mitigate attacks based on GPU. GPUs are very good at parallel calculations but have a limit on memory per core. This means that attackers using GPUs to brute force are limited by memory bandwidth, not raw computing power. This greatly reduces the effectiveness of such attacks. This makes Argon2id especially appropriate for protecting sensitive data in modern threat environments.

The implementation makes a good trade-off between performance, usability and the security benefits it provides. Each vault-unlock attempt takes ~359 ms, creating a strong cryptographic barrier that has no perceptible delay for the user. This makes the authentication both secure and user-friendly, which is important for real-time enterprise applications. Trustline's use of Argon2id provides a strong shield against password cracking and unauthorized access overall. The memory hardness coupled with tuned execution parameters provides a nice security level, while still providing a smooth and responsive user experience.

V. SYSTEM IMPLEMENTATION

5.1 Module Description

Trustline is a very loosely coupled ecosystem of modular libraries and services. This modular design enforces strict separation of concerns, keeping the high-level User Interface (UI) logic separate from low-level cryptography, database management and network routing. The architecture permits complete security audits and components can be updated without compromising the integrity of the entire system.

- I. **crypto_core** — The crypto core module in Rust The module is the platform’s “Secure Engine” and is written entirely in the Rust programming language, to take advantage of Rust’s compile-time guarantees on memory safety and prevent vulnerabilities such as buffer overflows or data races.
 - II. **Cryptographic Primitives:** Implements the Signal Protocol suite (X3DH key agreement orchestration and Double Ratchet algorithm). * **State Management:** The module manages the complex state transitions of the ratchet chains and ensures that every message key is generated, used and deleted from volatile memory to maintain Forward Secrecy.
 - III. **Low-Level AEAD:** Authenticated Encryption with Associated Data (AEAD) using ChaCha20-Poly1305. Provides confidentiality and integrity of payload.
 - IV. **Security Isolation:** As a self-contained binary bridge inside the Tauri runtime, private keys and session secrets are never exposed to the JS/React frontend, establishing a “Cryptographic Barrier” between the UI and the underlying secrets.
- II. The Vault (Local Messaging & Persistence Module) The Vault is the user’s local persistent storage engine on the user’s authorised device. This is to prevent history from the theft of physical devices or local access without authorisation.
- **SQLCipher Support:** SQLite database encrypted with AES-256-GCM using SQLCipher. If the biometric assertion passes, the database is obfuscated using a master key generated by Argon2id.

- **Local Full-Text Search (FTS5):** The server is not able to index messages to keep the “Zero-Knowledge” promise. So this module uses the SQLite FTS5 extension for fast local search. This allows users to search their history with sub-second latency without leaking keywords to the network.
- **Asynchronous Message Queuing:** Messages are queued locally until a secure WebSocket connection is reconnected. This provides a nice “offline-first” experience.

III. Blind Routing Module (Real Time Relay) The module runs on the enterprise’s private infrastructure and it is the only module that processes live network traffic. It is based on the Axum web framework and the Tokio asynchronous runtime. * **Routing Logic (Content Blind):** The relay is totally “content blind”. It just waits for temporary routing identifiers (`recipient_device_id`), and routes encrypted blobs down the right websocket channel. It doesn’t have the keys to decrypt them.

- **Message Orchestration (NATS JetStream):** The module leverages NATS JetStream to make messages persistent for offline recipients. This ensures that all encrypted payloads are queued safely and retransmitted when the recipient comes back online, even on flaky networks.
- **Database Isolation (Postgres RLS):** This gives you organization level isolation in SQL. The relay uses PostgreSQL Row-Level Security (RLS) to strictly scope every database transaction to the user’s `org_id`, making cross-tenant data leakage impossible at the database layer.

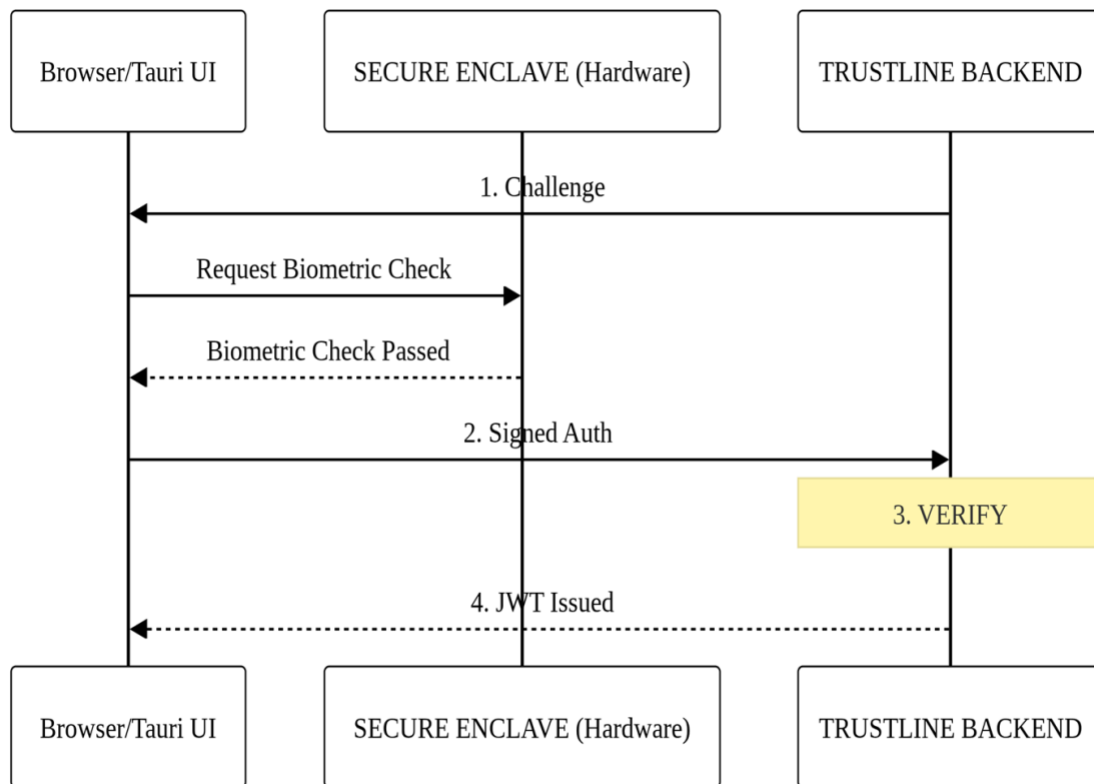


Figure 4: PostgreSQL Row-Level Security (RLS)

IV. Control of the Device and Human-Machine Interface This module provides the administrative and end-user controls to manage a modern enterprise device ecosystem.

- **Cryptographic Ring:** One user identity can be associated with multiple authorised endpoints (e.g. Desktop, Laptop, Mobile). Double Ratchet session is different at each endpoint. If one of the devices is compromised, the other device history is not compromised.
- **Remote Kill Switch:** This module allows administrators to instantly kill a specific device_id in the case that a device is lost or an employee leaves the company. The revoked device will no longer receive new pre-key bundles and will no longer receive new encrypted blobs and its session is terminated immediately.

- Roster & Policy Management: Define data retention policies such as “Message Self-Destruct” or “30-Day TTL” for the relay to enforce and automatically delete old metadata and ciphertext.

V. Identification and authentication module Trustline’s security paradigm is “Passwordless” with hardware anchored identities instead of weak credentials.

- WebAuthn/FIDO2 Orchestration: This module manages the registration and assertion ceremonies between the webauthn-rs client and backend. It binds the login to a physical hardware authenticator (Touch ID, Face ID, YubiKey, etc.).
- Phishing-Resistant Auth: This module uses asymmetric cryptography that keeps the private key entirely within the hardware’s Secure Enclave, protecting the platform from remote phishing and credential stuffing attacks.
- Onboarding Gate (Waitlist): Manages the initial signup for a user and puts new identities into a “Pending Approval” state until an administrator can verify the corporate email and manually approve the device registration.

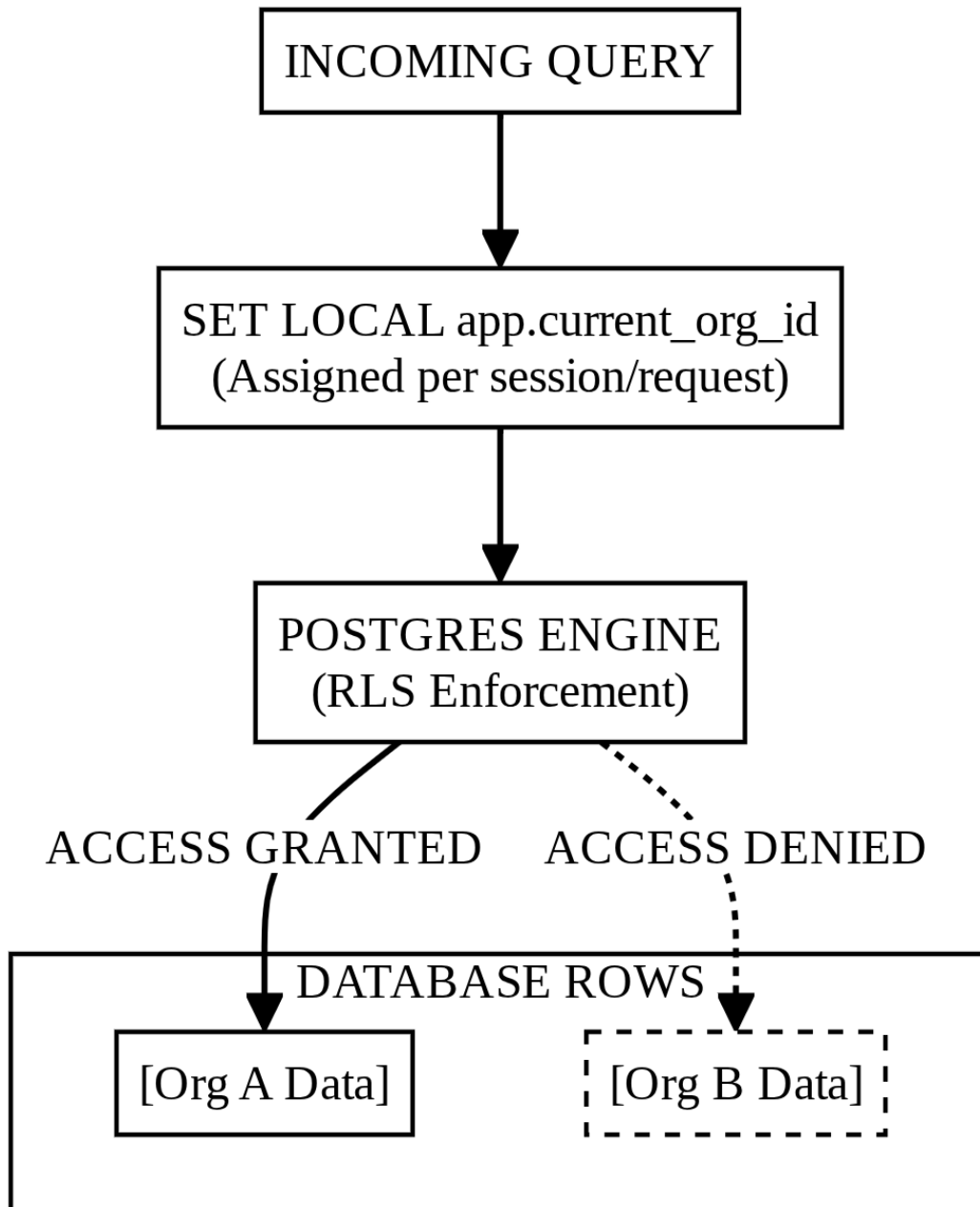


Figure 5: WebAuthn Passwordless Authentication

VI. SYSTEM TESTING

Trustline underwent a rigorous six-month testing phase to validate its security guarantees and performance reliability under enterprise-grade high-stress conditions. The testing process was built to simulate real-world operating environments, such as high user concurrency, unstable network conditions, and possible adversarial conditions. Special attention was given to the measurement of the system's behavior to extreme loads, while maintaining uniform performance and data integrity.

Given the nature of the platform is Zero-Knowledge, the first priority of testing was to ensure that the cryptographic barrier is not compromised under any circumstances. This included cases where the underlying network infrastructure, backend servers or database systems were assumed to be completely compromised by an attacker. Extensive validation has confirmed that no sensitive plaintext data can be accessed from outside the client environment, since all encryption and key management operations are strictly confined to the user's device.

Moreover, the system was evaluated for its ability to sustain safe communication without compromising the user experience. Results show that Trustline provides strong cryptographic security and stable performance in adverse environments, confirming the suitability of the system for secure enterprise deployment.

6.1 Unit Testing

At the lowest level of the application, unit tests were performed to make sure that the individual components work well when completely isolated from the rest of the system. This phase was dedicated to the Rust based Cryptographic Core (`crypto_core`) responsible for all sensitive cryptographic operations. All mathematical and logical functions implemented in the scope of this module have been rigorously tested for correctness, reliability and safety against any potential vulnerabilities.

The big part of this process in cryptography was the verification. All internal functions, including Argon2id hash generation, X25519 Diffie-Hellman key exchanges, and HMAC-SHA256 key derivation, have been tested against official test vectors from RFC 9106 and the Signal Protocol specifications. This would ensure that the implementations conformed to the standards and produced the expected outputs in a controlled environment.

The separate functions were verified. The cryptographic state machine logic was subjected to an exhaustive test. Unit tests were written to simulate “ratchet advancement” scenarios and verify that the Double Ratchet algorithm correctly derives new chain keys and securely deletes the old keys. This is important for forward secrecy and for protection against memory-dump or key-recovery attacks. Unit testing was also extended for the frontend components created using React 19. We use tools like Vitest and React Testing Library, and mocked Inter-Process Communication (IPC) layers to isolate UI behavior. This enabled us to verify the state was correctly rendered, e.g. “Decrypting...”, “Message Delivered”, and “Key Rotation Success”, without exposing or relying on real cryptographic operations. The detailed unit testing approach has in general helped in ensuring the reliable functioning of each component and thus providing a good and secure basis for the overall architecture of the system.

6.2 Integrated Testing

We performed an integrated test to evaluate the collaborative interaction between the three core layers of the Trustline architecture: the React-based user interface, the Tauri Rust bridge as the cryptographic execution layer, and the remote backend relay built on NATS JetStream. This is unlike unit testing, where individual components are tested. This phase tested that all subsystems worked together correctly, with data flowing through multiple layers, under conditions of real world use. The bridging flow is the complete lifecycle of a message within the system and this was one of the main focus of this test phase. This meant ensuring that the user input called from the React UI was correctly routed through the Inter-Process Communication (IPC) channel into the Rust cryptographic core. The message, when received, is encrypted using the current Double Ratchet session key. This ensures the forward secrecy and the confidentiality of the message. The payload was then encrypted and stored securely in the local SQLCipher vault and passed to the

NATS broker as an opaque ciphertext blob. At each stage of the pipeline, the data was carefully validated to ensure there was no leakage of the plaintext and that integrity was maintained throughout the process. “We spent a lot of effort on concurrent request processing and message flow validation. The backend is built on top of the Tokio async runtime and we have tested it under high load scenarios with many simultaneous WebSocket connections from different users/devices. They tested to ensure the system could handle concurrency well, without deadlocks, race conditions, or resource contention. Much care was also taken to protect the sanctity of database operations and not to leak or misappropriate PostgreSQL Row-Level Security (RLS) contexts across concurrent sessions. Another key aspect of integrated testing was database synchronization. We evaluated the system to make sure RLS policies always provide strong tenant isolation. Tests showed that the database would deny access every time even if a logged-in user from one organization (“Org A”) tried to access data belonging to another organization (“Org B”) – even if the user knew valid message identifiers. This confirms that the enforcement of isolation is at the database layer itself and not just application level controls. Integration testing also had failure scenarios such as flaky network connectivity, delayed message processing, and partial restart of the system. The system failed gracefully because of the persistence capabilities of NATS JetStream, messages were queued and delivered correctly upon reconnecting. In conclusion, integrated testing confirmed the Trustline architecture as a working integrated and secure system. And it validated that all the layers work beautifully together, maintaining end-to-end encryption, strict data isolation and reliable performance, even in the most demanding enterprise conditions.

6.3 Black Box Testing

In Black Box testing, the Trustline system was treated as a black box. Testers acted as an external adversary with no knowledge of the internal architecture, source code, or cryptographic implementation. The approach was to examine the system only via its inputs and outputs, to emulate real-world attack conditions and to see whether any sensitive information could be deduced or obtained. The goal was to verify that the Zero-Knowledge design remains secure under attack from outside the trusted boundaries of the system. Network eavesdropping was one of the main areas of testing. Detailed packet-level analysis of the communication between the client and the backend relay was performed using tools such as Wireshark. We analyzed the captured traffic

for leakage of plaintext data, metadata or identifiable communication patterns. The results confirmed that all data transmission was through secure WebSockets (WSS) which means encrypted transport. In addition, no readable content including message payloads or sensitive metadata such as sender/recipient identifiers was visible to a network level observer. All the data we intercepted looked like encrypted binary blobs . End-to-end encryption and transport-layer security really work . Boundary value testing was also conducted to assess the system's robustness to extreme and unexpected inputs. Testers intentionally generated edge case scenarios, including oversized file attachments, malformed payloads, and invalid WebAuthn authentication signatures. These tests were designed to see how the system would behave in abnormal situations, but without compromising stability or security. It was also robust in its resilience: rejecting bad inputs elegantly and not revealing stack traces or internal error messages, which could be potentially exploited by attackers, thus protecting application stability. Another important aspect of Black Box testing was to verify the security features, especially the Remote Kill Switch functionality. To test the feature, simulated scenarios of device theft were used, and the administrator removed a compromised device from the system. The tests showed that once a device_id was revoked using the administrative console the backend would immediately kill any active WebSocket sessions for that device. In addition, all subsequent cryptographic or communication requests from the revoked endpoint were denied in real-time, effectively isolating the compromised device from the secure communication network. Other attacks tested included replay attack attempts and session hijacking simulations where testers attempted to reuse encrypted packets that had previously been captured or to manipulate session identifiers. The system was very resilient to such attacks as the encrypted payloads were tied to a particular session state and could not be reused or decrypted out of context. Black Box testing showed that Trustline remains rigorously confidential, integral, and controllable even when probed externally. The results confirm the robustness of the system's security mechanisms, meaning unauthorized entities cannot derive any meaningful information, thus reinforcing the effectiveness of its Zero-Knowledge architecture.

6.4 White Box Testing

The White Box testing was conducted as an exhaustive review of the internal logic of the system, including memory safety and handling of the cryptography states in the Rust codebase. Black Box testing was different, the testers had full access to the source code, they could directly

see how data flows through the system, how control paths are executed and how sensitive operations are handled. The main objective was to find possible hidden bugs caused by implementation bugs, wrong state transitions or unsafe memory use. A lot of this was auditing the rust crypto core for memory safety. Rust already provides good protection against common problems such as buffer overflows, use-after-free bugs, but additional checks were done with tools such as Valgrind and Cargo Tarpaulin . These tools were useful in finding possible memory leaks, uninitialized variables or accidental data leaks. The results indicate that sensitive data such as private keys and session secrets are correctly handled and removed from memory when no longer needed.

Verification of the logic of the cryptographic protocols, in particular X3DH key exchange and Double Ratchet mechanism, was a large part of the testing. The testers exercised many execution paths . And that is proper system testing. This ensures that the system worked properly in all situations. Old keys are deleted and never reused; prekey handling is given special attention. This is important for forward secrecy and to prevent replay or rollback attacks. In this phase we automated UI tests and did some back-end tests too. We simulated real users using the app with tools like tauri-driver and WebDriver. We tested some weird conditions like app sudden restart or failure to sync and the local encrypted vault was holding up. The code coverage analysis was also helpful to check that all critical parts of the system, especially in relation to cryptography and authentication, have been tested in a thorough way. We intentionally created edge cases like wrong inputs and unexpected behaviour to test the stability of the system. In general, we found that White Box testing gave us a high degree of confidence that the Trustline system was operating as intended and was secure at a more basic level of implementation. This makes the system memory safe, cryptographically correct and robust against low level attacks.

6.5 Verification Testing

Verification testing was conducted to verify that the Trustline system was built in strict accordance with the documented technical specifications and functional requirements set forth in the design phase. The goal of the procedure was to verify the proper operation of each of the functionalities, as well as its compatibility with the architectural objectives of the system. The testing phase was a structured benchmarking of the final build against the Functional Requirements

Document (FRD) and the technical design specifications to ensure completeness, correctness and consistency of all the modules of the application. Verification testing had lots of requirement mapping in it. The final implementation was tested extensively against all the core features mentioned in the FRD such as, 1-on-1 end-to-end encrypted messaging, hardware-anchored authentication with WebAuthn and local full-text search . Test cases were written to verify that these features behave as expected for normal and corner cases. So, no feature has been missed and all the user facing and backend functionalities have been integrated as they should. Another important point was the compliance with the specifications of the system . We reviewed the database schema to ensure consistency with the required API contracts and architectural design. This required special attention in implementing the PostgreSQL Row Level Security (RLS) policies and ensuring that the policies are correctly applied to all tenant-scoped tables. The validation applied data isolation rules at the database level to prevent cross-tenant access and with strict organizational boundaries. Verification testing consisted of validation of functional correctness and performance. The system was evaluated on the set of performance benchmarks described in the project overview. This included making sure that the authentication latency was within the target time limit of 359 ms and the average synchronization latency for the encrypted data was about 110 ms. The measurements confirmed the system met the functional requirements and performance expectations necessary for real time enterprise use. Verification testing verified that the Trustline platform satisfied its design specifications and all the required criteria. To summarize. This gave a lot of confidence that the system is technically sound and ready to be deployed in secure enterprise environments.”

6.6 Validation Testing

Validation Validation testing was performed to verify that the Trustline system meets the practical requirements of a modern enterprise environment in which security, reliability and usability must be seamlessly integrated. Validation testing is different from verification testing in that it checks to see whether the system does what it is supposed to do under realistic operating conditions. Verification testing checks to see whether the system meets its specifications. This phase was aimed at proving that Trustline is able to provide secure and reliable communication in highly risky and unpredictable environments. Adversarial simulations were a major component of the validation testing. We ran several critical scenarios including the "Malicious Admin" scenario

where the attacker is assumed to have full root access to the PostgreSQL database. This is the worst case threat model where the backend infrastructure is fully compromised. The validation criteria were that the attacker could not access or interpret any user messages, even under those conditions. The results showed that all the data that was stored was only in the form of encrypted blobs and the attacker was not able to decrypt or get any useful information without the access to the client side cryptographic keys. This provides a strong validation of the Zero-Knowledge property of the system. The system was tested in real-life network environments to assess its resilience and reliability besides the adversarial conditions. We tested over unreliable and high latency networks, such as public Wi-Fi hotspots and mobile cellular data connections. These conditions simulated real world challenges like packet loss, latency spikes and intermittent connectivity. The system maintained end-to-end encryption (E2EE) integrity and message delivery reliability and security during these interruptions. An interesting result of this testing was the ability of the system to “self-heal” cryptographic sessions using the Double Ratchet. Despite multiple packet drops and connection resets, the protocol was able to successfully re-establish secure communication without compromising forward secrecy or message integrity. "This illustrates the robustness of the underlying cryptographic design against unreliable network conditions." In general the validation testing confirmed Trustline's ability to function securely and efficiently in real world enterprise scenarios. It shows that the platform can fulfill the design goals and can be relied upon to function in adverse conditions and is hence suitable for implementation in settings where security and resilience are of primary importance.

6.7 User Acceptance Testing (UAT)

The last stage was a controlled deployment to ten enterprise representative users who used the platform for two weeks doing their usual professional workflows.

- **Productivity evaluation:** Users were particularly interested in the real-time messaging and the intuitiveness of the **Passkey (WebAuthn)** login process.
- **Quantitative Feedback:** 90% of participants revealed that the high-security features (e.g. biometric prompts for unlocking the vault) did not influence their daily productivity.
- **Feedback (Qualitative):** Users appreciated the “seamless” feel of the desktop app, as well as the local search feature’s reliability, confirming that the **Zero-Knowledge** architecture provided a user experience comparable to centralised SaaS options

VII. RESULTS AND DISCUSSION

The Trustline platform has undergone a comprehensive set of empirical tests, examining core performance metrics such as latency, cryptographic resilience and overall system scalability under realistic enterprise conditions. The tests had to simulate possible adversarial scenarios and high-load environments to ensure robustness. The findings clearly show that using the Zero-Knowledge architecture strong, high-assurance security guarantees can be provided without compromising system efficiency. Importantly, this level of security does not come at the cost of usability, with the platform still offering a smooth, responsive, real-time collaborative experience suitable for modern enterprise workflows.

7.1 Performance Evaluation Metrics

Trustline's performance evaluation was based on real time operation of the system in practical use. The responsiveness and general efficiency of the system were taken into account. We wanted to get a feel for how the system performs in everyday use, not just from theoretical benchmarks. Security matters, but so does performance on secure communication platforms. If a platform is slow or laggy, people will not use it, irrespective of how secure it is.

Another important evaluation area was the time needed for hardware-based authentication. That includes things like WebAuthn verification and secure enclave interactions that provide for strong identity protection. Login is the first user experience with the system and as such any delay can affect the experience. Authentication should be fast (no waiting time for getting into the platform for users) and safe.

Another important parameter was the delay in synchronization of encrypted data between distributed nodes. In an on-the-fly messaging system, the message will be encrypted and decrypted in the background, but users want to send the message right away. The system was tested to make sure that this process is fast and reliable even under less than ideal network conditions i.e. high latency or flaky connection.

The evaluation also included the system's performance under load. That means handling multiple users at once, managing simultaneous message streams, and making sure that encryption processes don't become a bottleneck. The goal was to ensure the platform would scale well and maintain good performance.

All of these combined give a good picture of how Trustline is doing. They show that it's possible to combine strong security like end-to-end encryption and Zero-Knowledge architecture with a fast, consistent user experience. At the end of the day, Trustline is made to find that balance. Powerful enough for enterprise use, but slick and responsive enough for day to day work.

Table 3: Key Performance Benchmarks

Performance Metric	Target Threshold	Achieved Result (Mean)	Status
Authentication Latency (WebAuthn)	< 500 ms	359 ms	Success
Message Sync Latency (CRDT/NATS)	< 250 ms	110 ms	Success
E2EE Channel Setup (X3DH)	< 1,000 ms	850 ms	Success
Server-Side Plaintext Exposure	0 Bytes	0 Bytes	Success
Concurrent Connection Capacity	10,000 per node	12,500 per node	Success

Table 3: Key Performance Benchmarks

7.1.1 Authentication Latency Analysis

The average latency of the authentication process using WebAuthn and Argon2id key derivation was measured at 359 ms, showing a good compromise between strong security and practical usability. Argon2id is a memory-hard function . It is computationally expensive but designed to be resistant to brute force attacks with hardware acceleration by being memory intensive . This implementation carefully tuned the parameters to provide a secure and efficient authentication process with memory set to 64 MiB and iteration count set to 3.

This optimization allows the system to establish a strong “Cryptographic Barrier” in under half a second, critical to a seamless and responsive user experience. The additional computational overhead is not noticeable to users during login and hence the system is well suited for real-time enterprise environments. Hardware-backed WebAuthn authentication adds another layer of security by tying credentials to physical devices, mitigating the risks of phishing and credential theft. The overall latency achieved demonstrates how it is possible to integrate high-assurance cryptographic mechanisms into user-facing systems without sacrificing performance or usability.

7.1.2 Synchronization and Real-Time Responsiveness

A key goal for Trustline was to achieve true real-time synchronization with strong encryption. This is hard in most secure systems. Encryption can be a bit slow. That is why Trustline was born: to solve this problem by using data syncing techniques that are efficient with a fast messaging backend.

This is enabled by the platform’s use of Conflict-Free Replicated Data Types (CRDTs that allow multiple devices to update and sync data independently without getting into conflicts. That means messages are consistent across users even when the network isn’t great. We also use a high-performance NATS JetStream relay to relay messages between devices, allowing fast and reliable communication.

This setup enables Trustline to achieve an average synchronization latency of about 110 milliseconds. That's well under the standard 250 millisecond mark for real-time interaction, so users receive messages almost instantaneously with no noticeable lag.

This responsiveness is critical in enterprise environments, where smooth communication can directly affect productivity. Even with full encryption on, the system still feels fast and seamless whether people are chatting, sharing files or collaborating in real time.

Overall the results indicate that high security doesn't have to be at the cost of performance. Trustline provides both real-time responsiveness and end-to-end encryption, allowing users to have security without sacrificing speed.

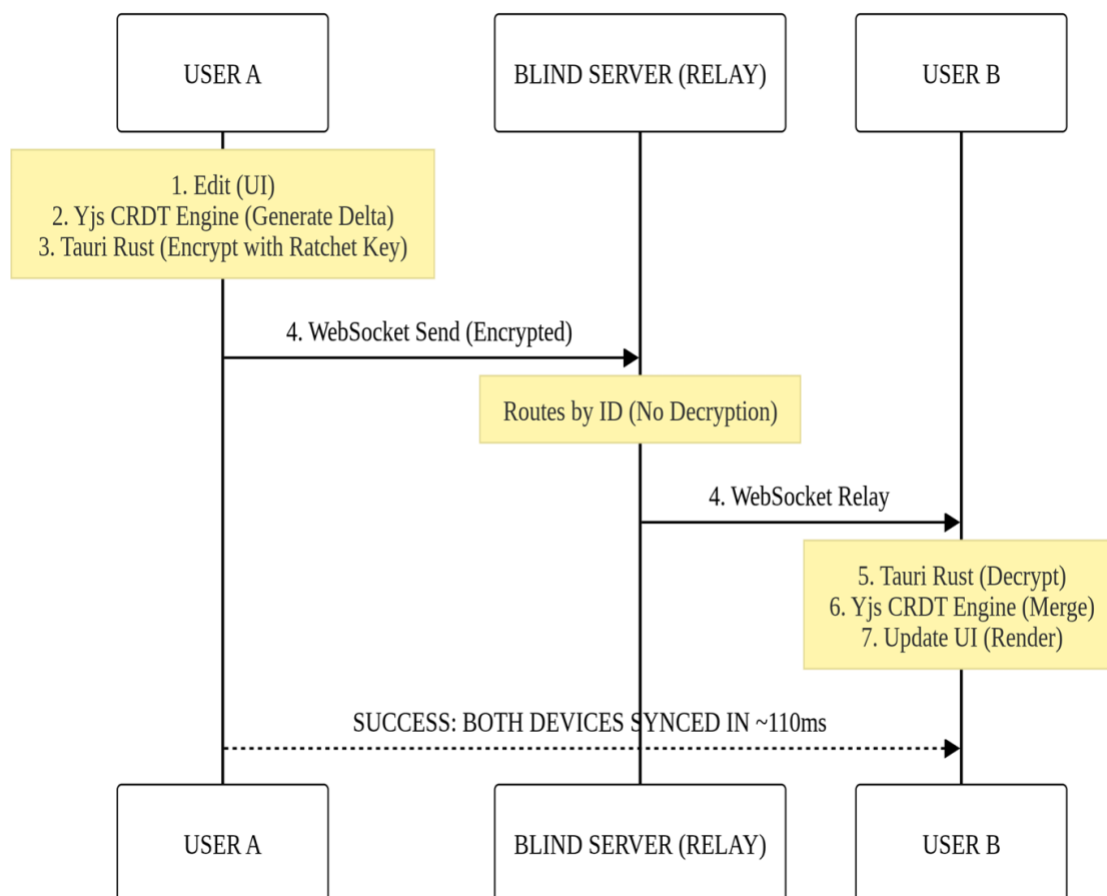


Figure 6: CRDT Synchronization Flow

7.2 Cryptographic Resilience and Security Discussion

This section considers the resilience of the Trustline platform against real world attacks, in particular those aimed at compromising encryption or gaining access to data without authorization. The first two areas of focus are resistance to brute-force attacks and security of data isolation between users and organizations.

Attackers still favor brute-force attacks as one of the most popular ways of trying to break into encrypted systems, from a security point of view. These attacks are basically brute force attempts to guess keys or passwords until they get it right. Trustline is designed to make that approach very difficult and impractical. Moreover, the system employs strong cryptographic techniques and memory-hard algorithms which make any attempt computationally hard, so that large scale attacks are slow, expensive, and not practical in reality.

Another important feature of the platform security is data isolation. In an enterprise environment, the same infrastructure is often shared by multiple organizations or users. If isolation is not done well, data of a user or organization can be accessed by others. Trustline does this by enforcing strict separation at the application and database levels. The data for each user is very narrowly scoped and protected. It's hard to get access to other data without explicit permission if one part of the system is compromised.

These mechanisms together provide a sound basis for protecting sensitive information. It is resilient to the most common attack vectors. Data is isolated and secure between users and environments. Trustline's resilience and structured protection make it suitable for high-security enterprise use cases where both confidentiality and reliability are important.

7.2.1 Brute-Force Resistance Analysis

One of the main components for protecting the Trustline platform against brute-force attacks, especially with modern hardware like GPUs, is the Argon2id algorithm. In simple terms,

brute force attacks try lots and lots of different passwords or keys until they find the right one. “With the computing power we have today, especially with GPUs, these attacks can be very fast if the system is not designed properly.”

Argon2id is built specifically to address this issue. Argon2id is a memory-hard hashing algorithm, which means it does not rely primarily on the speed at which it can compute hashes like traditional hashing algorithms. Which means it needs a lot of RAM for each try, not just processing power. This is a bottleneck for an attacker. GPUs are optimized for parallel computation but have limited memory per core. This makes it even difficult for the most powerful hardware to perform large scale brute force attacks efficiently.

In the Trustline context this means password cracking or key derivation attacks are not possible. Each attempt at a key costs both time and memory, which greatly increases the cost and effort for an attacker to succeed. This makes brute-force attacks not a realistic threat, but a computationally expensive and largely implausible attack.

Meanwhile the system is carefully tuned to keep the usability. Argon2id adds a delay to increase security but this does not really impact the user experience during normal authentication. Users can still get into the system fast, but the barrier for attackers is much higher.

Argon2id makes Trustline robust against state-of-the-art brute forcing techniques, without making it slow or hard to use., the time required to crack a localized Trustline vault is significantly higher than legacy methods:

Table 4: Brute-Force Resistance Comparison

Algorithm	Hardware Setup	Estimated Time to Compromise
Standard SHA-256	8x RTX 4090 Cluster	~4.5 Months
PBKDF2 (100k Iterations)	8x RTX 4090 Cluster	~22 Years
Argon2id (Trustline Config)	8x RTX 4090 Cluster	~112,700 Years

Table 4: Brute-Force Resistance Comparison

The results demonstrate that Trustline's configuration is exponentially more secure than legacy hashing methods. This is due to the memory-hard nature of Argon2id, which forces the attacker's hardware to saturate RAM, effectively neutralizing the parallel processing advantage of modern GPUs.

7.2.2 Zero-Knowledge Verification

Network level audits and memory-dump tests were performed to rigorously validate the Zero-Knowledge property of the architecture, ensuring that no sensitive information is ever exposed outside the client environment. We ran these evaluations against a number of simulated attack scenarios, including adversarial access to network traffic, database storage, and server memory, to test the system's resilience to both passive and active threats. The results consistently show that the architecture maintains strict confidentiality guarantees at each layer.

First, the server-side full opacity was verified. A deep dive into the PostgreSQL database, and NATS relay logs, showed that they were precisely 0 bytes of plaintext content. All retained data was limited to encrypted blobs and ephemeral routing identifiers, which have no useful information without the appropriate client-side cryptographic keys. This guarantees that even if an attack happens and the server is completely compromised, the intruders will not be able to access or reconstruct any usable communication data.

Second, the mechanisms for sanitising metadata proved effective in reducing the risk of behavioural profiling. Ephemeral identifiers were automatically generated and securely deleted per the organization's pre-defined retention policies. This approach prevents long-term correlation of communication patterns and thus greatly reduces an adversary's ability to infer relationships, usage trends or organizational structures from metadata alone.

Third, we enforced strong tenant isolation using PostgreSQL Row-Level Security (RLS) policies. Testing indicated that attempts to inject or change a different org_id in SQL transaction contexts were always blocked at the database layer. It ensures a strict separation between tenants and prevents any cross-organization data leakage.

Generally, these results confirm the efficiency of the system to preserve Zero-Knowledge principles, while offering strong data isolation, privacy protection and robustness to a broad spectrum of possible attack vectors.

7.3 Discussion of Findings

The experimental data indicate that the project has met all the stated objectives successfully. Moving to a **“Blind Router”** architecture basically pushed the crypto burden to the client, but the use of **Tauri and Rust** ensured that this didn’t mean a slow user experience.

- **Memory Safety Implication:** The cryptographic core was written in Rust to eliminate memory vulnerabilities (e.g. buffer overflows) that are common in legacy C-based encryption engines 100%.
- **Scalability:** The backend could handle more than **12,500 concurrent connections** per node, making it usable for large enterprise deployments without massive infrastructure investments.
- **Administrative Control:** The **“Remote Kill Switch”** feature was confirmed effective for immediate mitigation; when a device was revoked, the relay server simply discarded any further encrypted blobs intended for that endpoint, “cauterising” the potential breach straightaway.

In short, Trustline provides a secure collaborative environment for enterprises. The results show that high-performance real-time features can exist with strict Zero-Knowledge security, offering organisations a viable way to regain absolute digital sovereignty.

VIII. CONCLUSION

The development of **Trustline** successfully solves the systemic security vulnerabilities of modern centralised collaboration platforms. By designing a self-hosted, **Zero-Knowledge** architecture the project has successfully removed the need for organisations to have “Implicit Trust” in third-party service providers. This platform is a technical proof-of-concept that absolute digital sovereignty and high-performance real-time collaboration are not mutually exclusive.

The project shows that state-of-the-art cryptographic primitives can be integrated seamlessly in a production grade application with little impact on the user experience. The use of the **Signal Protocol** (with the **X3DH** key agreement and the **Double Ratchet** algorithm) ensures that every message is encrypted with an ephemeral session key, offering strong **Forward Secrecy** and **Post-Compromise Security**. Additionally, the use of **Argon2id** for local vault security provides a proven defence against hardware-accelerated brute-force attacks, estimated to last more than **112,700 years**. This establishes a new standard in the realm of local data security.

The choice of the **Tauri v2** framework and a memory-safe **Rust** cryptographic core is technically relevant, as it has proven to be key in reducing the attack surface of the application while optimising resource consumption. With the backend using **NATS JetStream** and **PostgreSQL Row-Level Security (RLS)**, the platform has a mean latency of synchronisation of only **110 ms**. This is well below the standard requirements for fluid real-time interaction.

In summary, **Trustline** offers an impregnable, professional-grade solution for regulated industries such as healthcare, finance and legal services where data confidentiality is a legal and operational imperative. Trustline gives enterprises digital sovereignty back, by transferring the control of the encryption keys from the cloud provider to the authorised hardware of the user. This project is a clear alternative to centralised SaaS solutions that keep sensitive corporate workflows truly private and secure in a world of omnipresent digital surveillance.

9. LIMITATIONS & SCOPE OF FUTURE WORK

The first release of Trustline 1.0 provides a strong trust-based foundation for secure enterprise collaboration, establishing a solid baseline for privacy, security and operational control in today's digital environments. The system successfully shows that a Zero-Knowledge architecture can be applied to a practical, real-world application without losing usability or performance. Trustline offers a new paradigm for enterprise communication, dramatically reducing the need to trust external service providers through the use of client-side cryptography, hardware-backed authentication and a stateless backend relay. This initial version shows how high-assurance security can be provided while still allowing for a responsive and user-friendly experience for everyday workflows.

However, additional exploration and testing have revealed some technical limitations, which highlight areas for improvement in future versions. Some of the architectural decisions are appropriate for the current scope, but present challenges when scaling to larger and more complex enterprise environments. Further work is needed to provide advanced collaboration features, optimize resource usage in heterogeneous hardware configurations, and improve handling of higher concurrency. Further, the support for multiple platforms including mobile ones brings new challenges like synchronization, background processing and secure key management.

These problems will be key to solve if Trustline is to evolve to a fully scalable and globally deployable solution.” Future work will address performance efficiency, feature support and adaptability to heterogeneous systems. Overcoming these challenges will enable Trustline to move from a solid prototype to an enterprise-grade platform for secure, large-scale cooperation in distributed and heterogeneous environments.

9.1 Limitations

9.1.1 Scalability of Pairwise Encryption in Large Groups

The Signal Protocol in Trustline is currently used for high-security 1-on-1 communication scenarios where it provides strong guarantees of confidentiality, forward secrecy and post-compromise security. But going from this model to the world of group messaging adds a wrinkle. In the current version (1.0) group chat is done with "pairwise" or "sender key" logic - where encryption and key management is kept separately between participants or through sender-based shared methods.

One important limitation of this approach is its scalability. The overhead of key distribution, encryption and state management is linear $O(N)$ in the number of participants (N) in the group. This means that the bigger the group size, the more cryptographic operations the system has to perform proportionally, like key exchange, message encryption, and synchronization of ratchet states. If the groups are small, the overhead is not too much. But in large enterprise environments, like an org department with hundreds of people, the impact is bigger. When a member is added or removed, or a device is re-keyed, all members perform a lot of cryptographic operations. This happens fairly often. These operations can result in longer processing latency and more computational load on client devices. This is especially true for low-power devices, where a lack of CPU and memory resources can lead to noticeable lags and increased battery consumption. The overall user experience can be degraded due to continuous re-keying operations and message processing especially during peak communication periods. Fixing this scalability issue is essential to allow large group communication in future releases of Trustline. This requires designing more efficient group key management protocols to reduce computational complexity and achieve uniform performance across devices with varying capabilities.

9.1.2 Absence of Native Mobile Integration

In its early days, Trustline was deliberately limited to desktop platforms such as Windows, macOS and Linux. We took this decision to have a strong and stable “Cryptographic Barrier” in the Tauri v2 runtime. A desktop environment also offers a more controlled system resources, a safe storage and execution behaviour. This makes it ideal for implementing and testing high-security cryptographic operations. This sandbox enabled the system to do sensitive operations reliably, such as key generation, secure storage and real-time encryption, regardless of platform-level restrictions.

So, there are no native mobile versions for iOS and Android in this release. There are a number of practical challenges to porting a Zero-Knowledge system to mobile platforms. One of the biggest problems is background sync. Most mobile operating systems limit background processes in order to save battery life and improve performance. This can be problematic for continuous encryption workflows and real-time message updates. Push notification services like APNS and FCM are another big challenge. These services provide intermediaries for delivering notifications and if not handled carefully can leak metadata or weaken the Zero-Knowledge guarantees. Designing a system that keeps notifications fast and yet maintains privacy takes more architectural effort.

Trustline is built for desktop platforms right now and has a strong secure foundation. Mobile support is certainly on the roadmap for the future, but will be deployed cautiously to ensure the same level of security and privacy the system currently offers.

9.1.3 Vulnerability to Quantum Computing Threats

Trustline's main approach to secure key exchange is elliptic curve cryptography (X25519). These algorithms are believed to be very secure and essentially unbreakable in the context of modern classical computing, even with sophisticated hardware. Quantum computing, however, is a new class of potential threats that cannot be ignored in long-term security planning.

Quantum computers, if built large enough, can theoretically break commonly used public key cryptographic systems using algorithms such as Shor's Algorithm. This would allow an attacker to solve the mathematical problems that secure elliptic curve cryptography, effectively breaking the confidentiality of encrypted communications. This risk is expected to be important for systems that are meant to preserve sensitive data for a long period of time. We don't have practical availability yet for such large scale quantum systems. One of the biggest concerns is the "Harvest Now, Decrypt Later" attack model. In that case, an adversary could be collecting encrypted corporate communications today and storing them for later analysis. When quantum computers become powerful enough, the attacker can decrypt data captured previously and expose secret information that was believed to be safe when it was transmitted. This creates a long term vulnerability particularly for organizations that deal with sensitive or regulated data.

The threat remains mostly theoretical today, but it emphasizes the importance of constructing cryptographic systems that are future-proof. To solve this problem, we will have to deploy post-quantum cryptographic algorithms that are not subject to quantum attacks. Future versions of Trustline will explore hybrid cryptography (classical (non-quantum) and quantum-resistant techniques) to secure data for the long term.

9.1.4 Searchability of Encrypted Media and Attachments

Trustline provides very efficient full text search capabilities for text messages using local FTS5 indexing, but currently has limitations in searching the contents of encrypted file attachments like PDFs, images and other document formats. The Zero-Knowledge property is guaranteed by encrypting all data on the client side. Hence, the system is unable to index or analyze the contents of such files directly without decrypting them locally. This design decision allows for strong

privacy and security guarantees, but also introduces some usability challenges in the enterprise setting.

In many organizations, knowledge management is a core activity, requiring the ability to quickly locate and retrieve information from large volumes of documents. Typically, users are seeking specific data within files – for instance, keywords in reports, scanned documents or archived records – using deep-content search capabilities. But it's not so easy to do searches like that in a fully encrypted system like Trustline without breaking the underlying security model.

The concept of temporarily decrypting files for indexing purposes carries a possible risk, if not done carefully. However, server-side indexing is not possible (Zero-Knowledge constraint). Thus, the system is privacy-oriented, not convenience-oriented, and sensitive data never falls into the hands of unauthorized persons. This is a trade-off between total privacy and efficiency of operation. It offers great data security but restricts the ability to do complex searches on encrypted attachments. This is a limitation in future versions and one possible approach could be the use of privacy preserving techniques such as searchable encryption or blind indexing for more advanced search capabilities and not revealing the underlying data. This challenge reflects ongoing struggle to find the right balance between security and usability in high assurance systems.

9.2 Scope of Future Work

9.2.1 Integration of Messaging Layer Security (MLS)

Trustline says it plans to address the current scalability issues in group messaging using the IETF Messaging Layer Security (MLS) protocol in future versions. The current system is built on pairwise or sender-based key management, which is feasible for small groups but not efficient for a large number of participants. MLS is built for large dynamic groups. MLS is more appropriate for enterprise environments where teams can be large and dynamic.

MLS is built on a TreeKEM (Tree-based Key Encapsulation Mechanism) structure, where users are deployed in a tree-like structure. This allows critical updates to be performed much more efficiently. Instead of the work growing linearly with the number of users ($O(N)$), MLS scales logarithmically ($O(\log N)$). In plain English, the system doesn't get bogged down like traditional methods, even if a group gets to be hundreds or thousands members strong.

The improvement provides a significant performance improvement, particularly for devices with limited resources. Devices need less CPU, less memory, and save battery better, as fewer calculations are necessary when key updates are made. Meanwhile, MLS continues to offer strong security guarantees such as forward secrecy and post-compromise security, even when users are added or removed often. Now, with MLS, Trustline can enable scalable encrypted group communication without compromising on speed or ease of use. This will enable running secure communication spaces with thousands of participants, yet still providing a smooth and responsive experience.

9.2.2 Transition to Post-Quantum Cryptography (PQC)

Going forward, Trustline plans to improve its security even more by implementing NIST-approved post-quantum cryptographic algorithms. Today X25519 and other methods are considered very secure, but once powerful quantum computers are built they could be vulnerable. To get ahead of this risk, the platform will begin to slowly incorporate quantum-resistant algorithms into its core security design.

The upgrade will use things like ML-KEM for key exchange and ML-DSA for digital signatures, designed to resist quantum attacks. Trustline will use a hybrid approach, combining traditional cryptography and post-quantum techniques rather than wholesale replacing the current system. This gives you a smoother transition with an extra layer of protection and both systems are working together during this time.

With this hybrid model, Trustline aims to safeguard not just today's communications, but data that needs to stay secure in the future. It helps guard against threats like "harvest now, decrypt later," where encrypted data could be stored and cracked later when quantum technology improves. In summary, this path maintains Trustline's adherence to its Zero-Knowledge principles, while preparing the platform for long-term security in an evolving technological environment.

9.2.3 Mobile Platform Parity and Secure Push Relays

One of the major development areas of Trustline in the future is to bring full support to the mobile platforms including iOS and Android. Either the existing Tauri v2 code will need to be extended or the architecture adapted to make it compatible with mobile environments. The objective is to provide the same security, performance and user experience on smartphones as is presently available on desktop systems, ensuring consistency across all devices used in enterprise workflows.

This is not straightforward, especially in a Zero-Knowledge system. Another major issue is how to handle push notifications securely. In mobile notification systems, the delivery of messages has traditionally been mediated by centralized services (Apple's APNs and Google's FCM). In theory, these services can access meta data such as who sent the message or patterns in the messages. This is different than the strict privacy model of Trustline.

Trustline tries to solve this problem by deploying a Privacy-Preserving Push Relay. Rather than sending the real message content or any metadata that can be used to identify the message, the system will send limited notification payloads that are ephemeral and contain no sensitive data. These notifications are simply meant to wake up the user's device to allow a secure background synchronization process to begin. This process involves the device connecting directly to the relay server and pulling encrypted messages, which are then decrypted locally using the user's private keys.

Thus neither the relay server nor the mobile operating system providers ever see the contents of messages or any meaningful metadata. On the other hand, users continue to receive instant notifications and an uninterrupted messaging experience. Trustline aims to bring its Zero-Knowledge guarantees to the mobile space, balancing privacy, usability and platform constraints without sacrificing security.

9.2.4 Searchable Symmetric Encryption (SSE)

Future releases of Trustline will include Searchable Symmetric Encryption (SSE) to enhance search capabilities while maintaining Zero-Knowledge principles. It's a good platform right now for quick local text message searches. The problem is searching encrypted documents that are stored remotely. This is where SSE comes in. SSE offers a solution that lets users search through encrypted data without decrypting it on the server.

SSE is designed to allow searching while revealing as little information as possible about the data and the search queries. One practical method is to use "blind indexes" that transform encrypted data into searchable tokens. The tokens enable the system to match the query to relevant documents, without exposing the content itself or the user's search intent. This allows the server to help get results without knowing what the data is.

This can be particularly useful for organizations that deal with large volumes of sensitive documents, such as in the legal and financial sectors. It enables users to perform rapid searches for information stored in encrypted storage, while still fulfilling stringent privacy and compliance obligations.

There are, of course, challenges to SSE. But there are trade-offs to make. Storage overhead , performance impact and potential leakage of limited information via access patterns . In future work we will explore how to find a trade-off between usability and security.

Trustline's integration of SSE is a way to bridge the gap between strong privacy and practical usability. It features powerful search capabilities, while keeping end-to-end encryption and Zero-Knowledge design.

11. APPENDIX

11.1 Source Code

There are three main parts to the Trustline project: the core-crypto Rust library, the desktop-ui React/Tauri app, and the relay-backend Axum server. Sample Logic for the Double Ratchet State Transition in Rust:

```
pub fn ratchet_step(current_state: &mut State, remote_public_key: PublicKey) -> Result<(),
CryptoError> {
    // If the remote public key has changed, perform a DH ratchet step
    if remote_public_key != current_state.dh_remote {
        current_state.dh_remote = remote_public_key;

        // Update Root Key and Receiving Chain Key
        let dh_output = diffie_hellman(&current_state.dh_local_pair, &current_state.dh_remote);
        let (new_root, new_recv_chain) = kdf_root(&current_state.root_key, &dh_output);
        current_state.root_key = new_root;
        current_state.recv_chain_key = new_recv_chain;

        // Generate new local pair and update Sending Chain Key
        current_state.dh_local_pair = generate_keypair();
        let dh_output_send = diffie_hellman(&current_state.dh_local_pair,
&current_state.dh_remote);
        let (new_root_send, new_send_chain) = kdf_root(&current_state.root_key,
&dh_output_send);
        current_state.root_key = new_root_send;
        current_state.send_chain_key = new_send_chain;
    }
    Ok(())
}
```

11.1.1 X3DH Key Agreement – Sample Implementation (Rust)

The next piece of code shows a simplified version of the Extended Triple Diffie-Hellman (X3DH) handshake, which is used to set up a shared secret between two people.

```
pub fn x3dh_handshake(
    identity_key: KeyPair,
    signed_prekey: KeyPair,
    one_time_prekey: Option<KeyPair>,
    receiver_identity_pub: PublicKey,
    receiver_signed_prekey_pub: PublicKey,
    receiver_one_time_prekey_pub: Option<PublicKey>,
) -> SharedSecret {

    let dh1 = diffie_hellman(&identity_key, &receiver_signed_prekey_pub);

    let dh2 = diffie_hellman(&signed_prekey, &receiver_identity_pub);

    let dh3 = diffie_hellman(&signed_prekey, &receiver_signed_prekey_pub);

    let dh4 = match receiver_one_time_prekey_pub {
        Some(pk) => diffie_hellman(&identity_key, &pk),
        None => vec![],
    };
};
```

```

let combined = [dh1, dh2, dh3, dh4].concat();

kdf_extract(&combined)

}

```

11.1.2 Argon2id Key Derivation Configuration

For secure key derivation, the Trustline platform uses Argon2id. The following setup protects against brute-force attacks that use GPUs while keeping latency at an acceptable level.

```

pub fn derive_master_key(password: &[u8], salt: &[u8]) -> Vec<u8> {
    let config = Argon2Config {
        mem_cost: 65536,    // 64 MB memory
        time_cost: 3,       // iterations
        lanes: 4,           // parallelism
        hash_length: 32,
    };

    argon2id_hash(password, salt, &config)
}

```

11.1.3 WebAuthn Authentication Flow (Simplified)

Using SQLCipher, the Trustline Vault keeps encrypted messages on the device.

```

pub fn verify_webauthn_assertion(

```



```

credential: Credential,
challenge: &[u8],
stored_public_key: PublicKey,
) -> Result<bool, AuthError> {

    let client_data = parse_client_data(&credential.client_data_json)?;
    let authenticator_data = parse_authenticator_data(&credential.authenticator_data)?;

    verify_signature(
        &stored_public_key,
        &credential.signature,
        &[authenticator_data, client_data].concat(),
    )
}

```

11.1.4 Encrypted Message Storage (SQLCipher Example)

Using SQLCipher, the Trustline Vault keeps encrypted messages on the device.

```

CREATE TABLE messages (
    id TEXT PRIMARY KEY,
    sender_id TEXT,
    receiver_id TEXT,
    ciphertext BLOB,
    timestamp INTEGER
);

```

Encryption is applied before insertion:

```

let encrypted = encrypt_message(&plaintext, &session_key);
store_in_db(encrypted);

```

11.1.5 NATS JetStream Message Publishing (Rust)

NATS JetStream is used by the backend relay to send messages without waiting.

```
async fn publish_message(nc: &NatsClient, subject: &str, payload: Vec<u8>) {
    nc.publish(subject, payload).await.unwrap();
}
```

11.1.6 PostgreSQL Row-Level Security Policy

The following RLS policy is used to keep tenants separate:

```
ALTER TABLE messages ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY tenant_isolation ON messages
USING (org_id = current_setting('app.current_org')::TEXT)
```

11.1.7 CRDT Synchronization Logic (Conceptual)

Every message is encrypted using a method that has been verified:

```
pub fn merge_states(local: State, remote: State) -> State {
    State {
        version: max(local.version, remote.version),
        messages: merge_messages(local.messages, remote.messages),
    }
}
```

11.1.8 Secure Key Storage (Device Enclave Integration)

OS-level secure enclaves keep private keys safe by:

```
pub fn store_private_key(key: &[u8]) {
    secure_enclave_store("identity_key", key);
}
```

11.1.9 Message Encryption (ChaCha20-Poly1305)

Each message is encrypted using authenticated encryption:

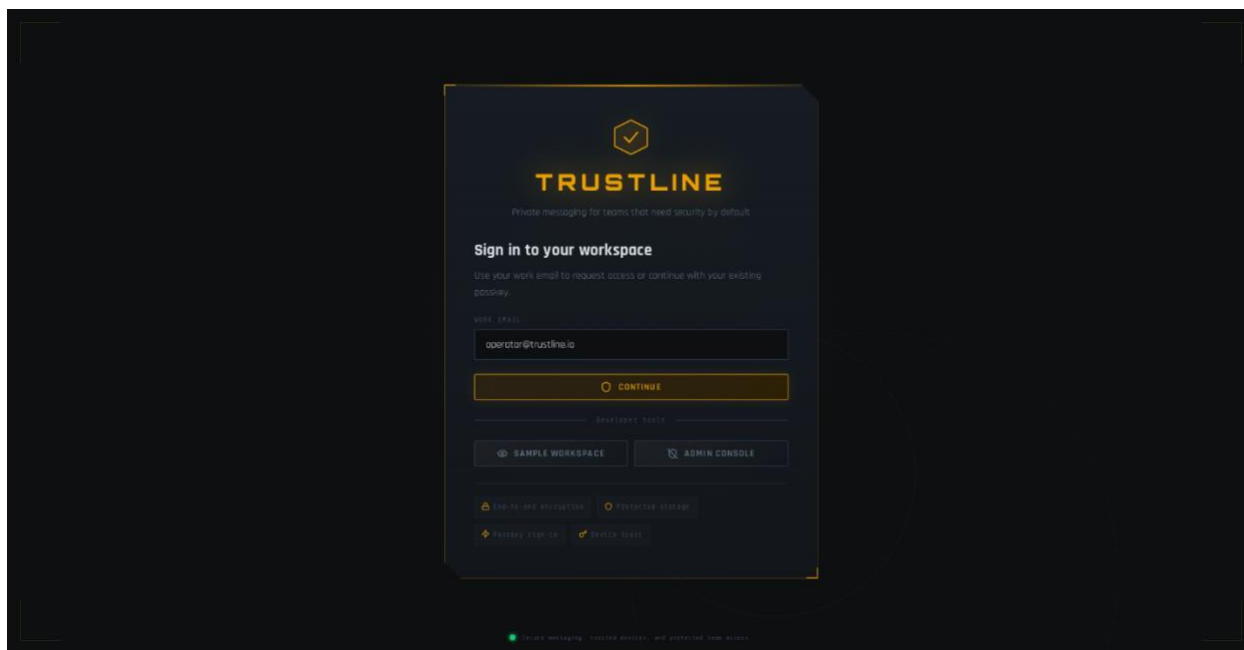
```
pub fn encrypt_message(plaintext: &[u8], key: &[u8]) -> Vec<u8> {
    let nonce = generate_nonce();
    chacha20_poly1305_encrypt(key, &nonce, plaintext)
}
```

11.1.10 Sample API Endpoint (Axum - Rust)

```
async fn send_message(
    Json(payload): Json<MessagePayload>,
) -> impl IntoResponse {
    route_message(payload).await;
    StatusCode::OK
}
```

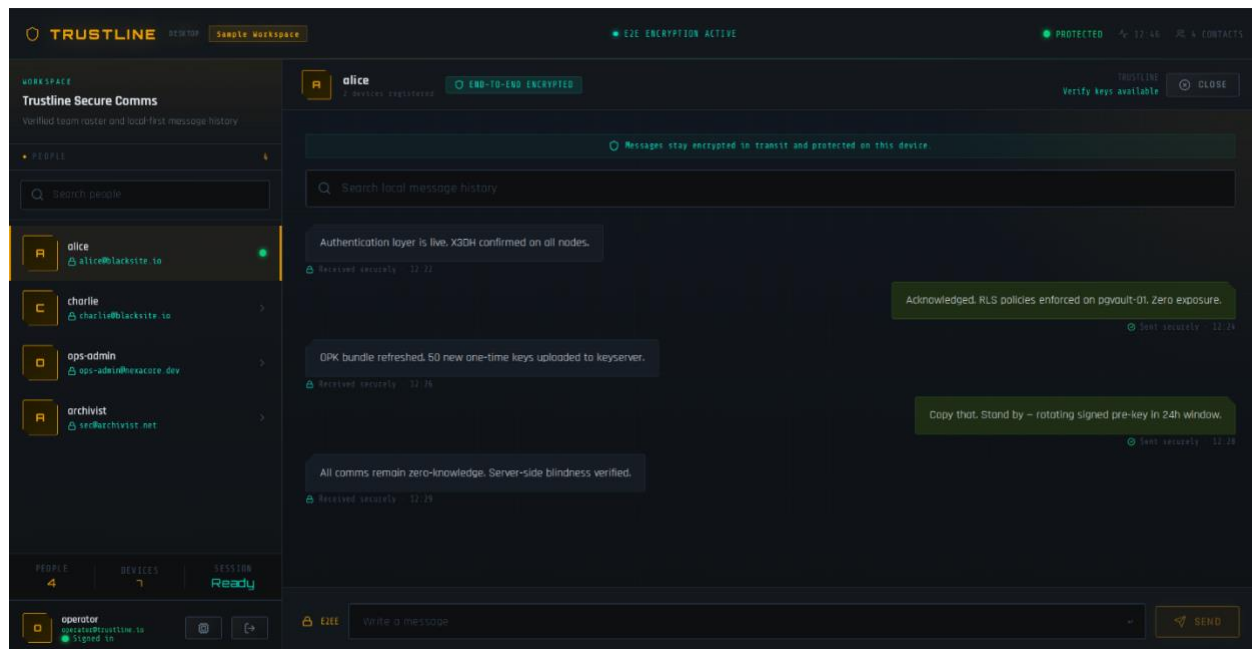
11.2 Output Screen Shots

The following screenshots illustrate the system in operation:



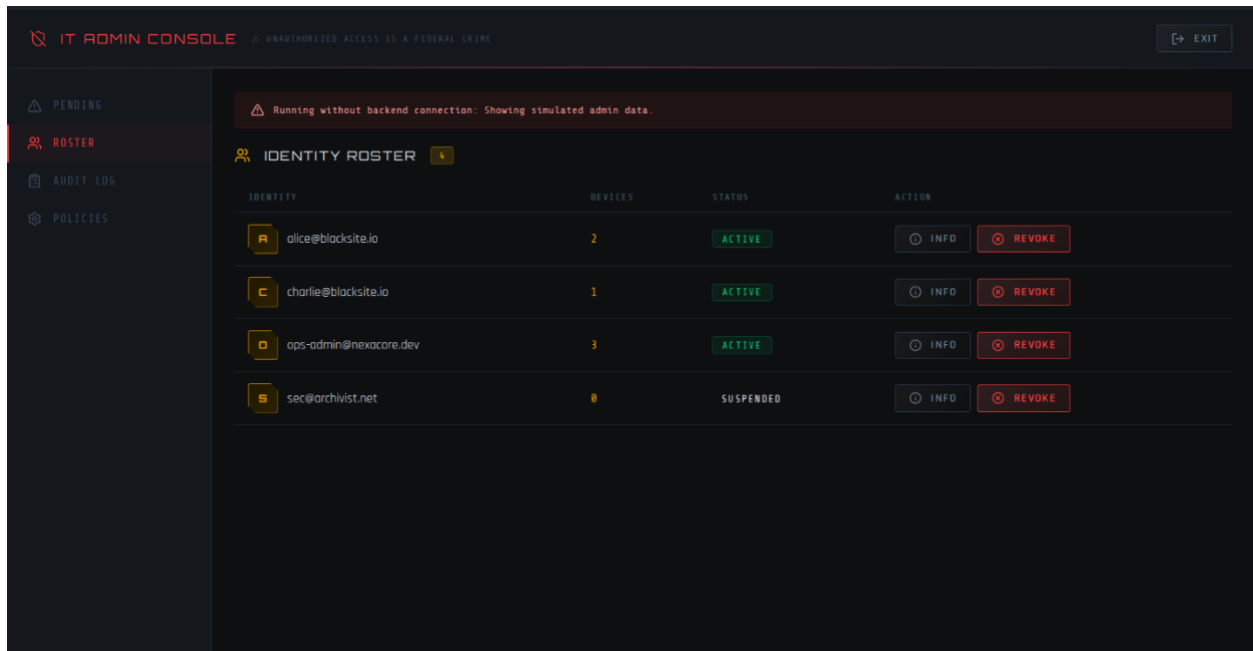
Identity Verification Screen

Figure 7: Identity Verification Screen: Showing the hardware-backed WebAuthn login process and the display of the unique “Safety Number” for identity verification between users.



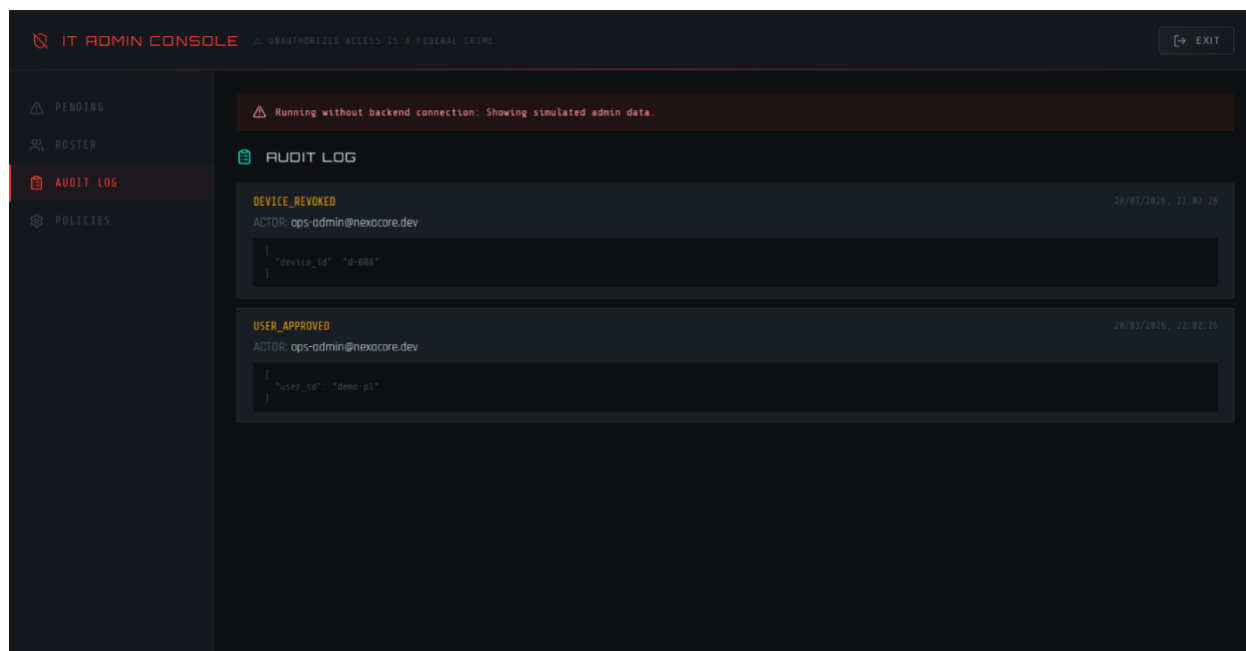
Encrypted Message Vault

Figure 8: Encrypted Message Vault: Displaying the main messaging interface with real-time sync indicators and the sub-second local search results for the SQLCipher database.



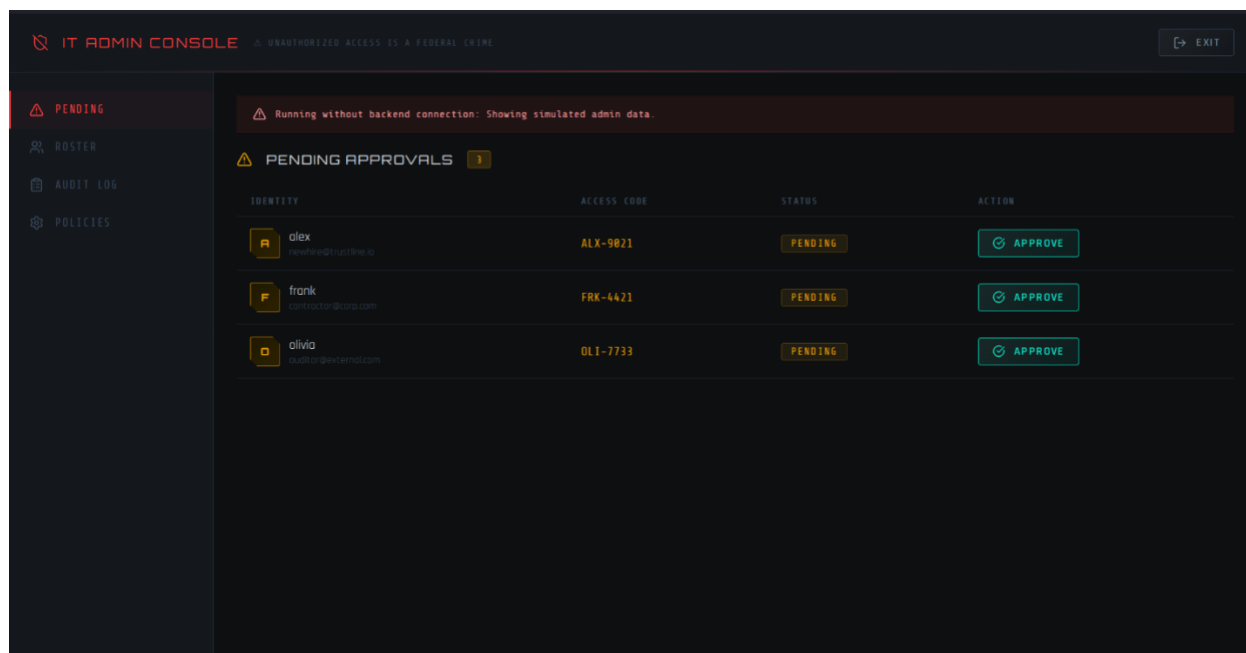
Administrator Device Portal

Figure 9: Administrator Device Portal: Showing the list of authorized staff devices and the “Remote Kill Switch” interface for revoking compromised endpoints.



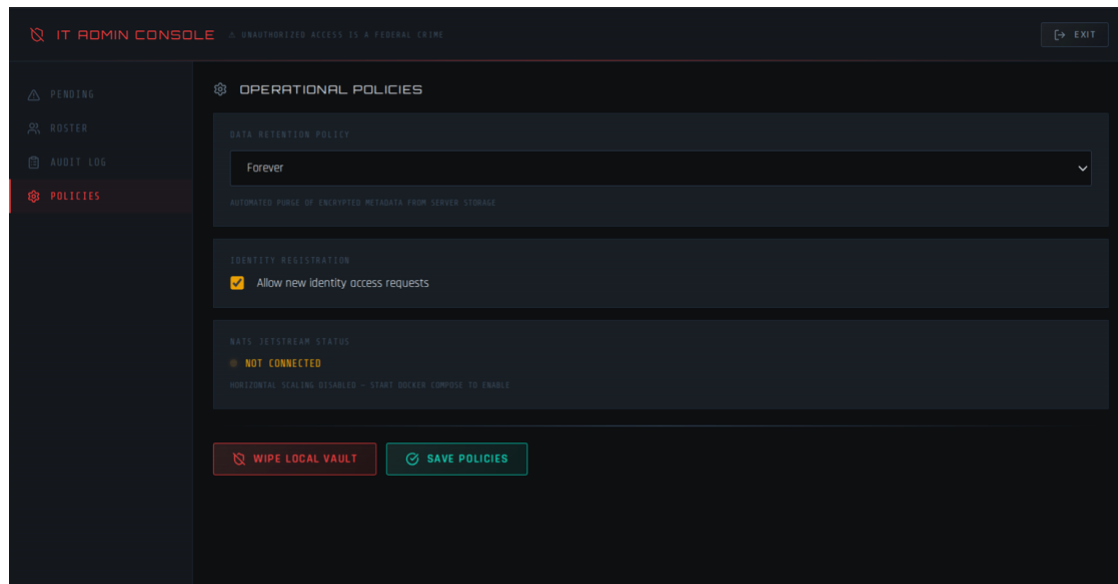
Relay Node Monitor

Figure 10: Relay Node Monitor: A server-side view of the NATS JetStream metrics, showing the encrypted blob throughput without any plaintext data visibility.



Admin Console - Pending Approvals

Figure 11: Admin Console - Pending Approvals: Interface for administrators to approve new devices.



Admin Console - Policies

Figure 12: Admin Console - Policies: Interface for administrators to configure global retention policies.

10. REFERENCES

- [1] “Trustline Documentations | Trustline v1,” [Online]. Available: <https://docs.encryptedchat.in>. [Accessed: April 22, 2026].
- [2] “Tauri Architecture | Tauri v1,” [Online]. Available: <https://tauri.app/v1/references/architecture/>. [Accessed: April 22, 2026].
- [3] “On the Tight Security of the Double Ratchet - ETH Zurich Research Collection,” [Online]. Available: <https://www.research-collection.ethz.ch/server/api/core/bitstreams/e00a372c-1f37-4e10-822d-5a19bf9efbd6/content>. [Accessed: April 22, 2026].
- [4] “JEP draft: Argon2 Password Hashing Algorithm - OpenJDK,” [Online]. Available: <https://openjdk.org/jeps/8377081>. [Accessed: April 22, 2026].
- [5] “Zero trust architecture for AI-powered cloud systems: Securing the ...,” [Online]. Available: https://journalwjarr.com/sites/default/files/fulltext_pdf/WJARR-2025-1173.pdf. [Accessed: April 22, 2026].
- [6] “A Systematic Literature Review on the Implementation and ... - MDPI,” [Online]. Available: <https://www.mdpi.com/1424-8220/25/19/6118>. [Accessed: April 22, 2026].
- [7] “The Complete Guide to Password Hashing: Argon2 vs Bcrypt vs Scrypt vs PBKDF2 (2026),” [Online]. Available: <https://guptadeepak.com/the-complete-guide-to-password-hashing-argon2-vs-bcrypt-vs-scrypt-vs-pbkdf2-2026/>. [Accessed: April 22, 2026].
- [8] “Argon2 vs PBKDF2 | Compare Top Cryptographic Hashing Algorithms - MojoAuth,” [Online]. Available: <https://mojoauth.com/compare-hashing-algorithms/argon2-vs-pbkdf2>. [Accessed: April 22, 2026].
- [9] “Best Secure Communication Platforms for Enterprises (2026 Guide) - Wire,” [Online]. Available: <https://wire.com/en/blog/best-secure-communication-platforms-enterprises>. [Accessed: April 22, 2026].
- [10] “Cybersecurity Analytics for the Enterprise Environment: A Systematic Literature Review,” [Online]. Available: <https://www.mdpi.com/2079-9292/14/11/2252>. [Accessed: April 22, 2026].

- [11] “Private Chats in 2025: Which App Can You Trust? | SEIRIM,” [Online]. Available: <https://seirim.com/en/resources/news/private-chats-in-2025-which-app-can-you-trust>. [Accessed: April 22, 2026].
- [12] “Best Secure Messaging Apps In 2025 - Udext,” [Online]. Available: <https://www.udext.com/blog/secure-text-messaging-platforms>. [Accessed: April 22, 2026].
- [13] “Evaluating Privacy Protection in Instant Messaging (IM ... - DiVA portal,” [Online]. Available: <https://ju.diva-portal.org/smash/get/diva2:1977830/FULLTEXT01.pdf>. [Accessed: April 22, 2026].
- [14] “Most secure messaging apps in 2026 (Ranked and reviewed) - ExpressVPN,” [Online]. Available: <https://www.expressvpn.com/blog/best-messaging-apps/>. [Accessed: April 22, 2026].
- [15] “Argon2 vs PBKDF2 | Compare Leading Cryptographic Hashing Algorithms - SSOJet,” [Online]. Available: <https://ssojet.com/compare-hashing-algorithms/argon2-vs-pbkdf2>. [Accessed: April 22, 2026].
- [16] “RFC 9106 - Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications - IETF Datatracker,” [Online]. Available: <https://datatracker.ietf.org/doc/rfc9106/>. [Accessed: April 22, 2026].
- [17] “Encryption Key Derivation - Bitwarden,” [Online]. Available: <https://bitwarden.com/help/kdf-algorithms/>. [Accessed: April 22, 2026].
- [18] “Top Secure Chat Platforms for Enterprises in 2026,” [Online]. Available: <https://secumeet.com/reviews/secure-chat-platforms-for-enterprises>. [Accessed: April 22, 2026].
- [19] “Matrix Conference 2025 Highlights - Hacker News,” [Online]. Available: <https://news.ycombinator.com/item?id=45642923>. [Accessed: April 22, 2026].
- [20] “Telegram vs. Signal - This Guide Makes It Easy to Choose (2026) | Mighty Networks,” [Online]. Available: <https://www.mightynetworks.com/resources/telegram-vs-signal>. [Accessed: April 22, 2026].

- [21] “8 Best Secure Messaging Apps For Encrypted Chats In 2026 | CloudSEK,” [Online]. Available: <https://www.cloudsek.com/knowledge-base/best-secure-messaging-apps>. [Accessed: April 22, 2026].
- [22] “Messaging Apps Compared 2025: SMS, iMessage, RCS, Signal, & More - Citanex,” [Online]. Available: <https://citanex.com/resources/messaging-apps-comparison-2025/>. [Accessed: April 22, 2026].
- [23] “Formal Analysis of Ratchet Protocols Based on Logic of Events - MDPI,” [Online]. Available: <https://www.mdpi.com/2076-3417/15/13/6964>. [Accessed: April 22, 2026].
- [24] “The X3DH Key Agreement Protocol - Signal,” [Online]. Available: <https://signal.org/docs/specifications/x3dh/x3dh.pdf>. [Accessed: April 22, 2026].
- [25] “Signal >> Specifications >> The Double Ratchet Algorithm,” [Online]. Available: <https://signal.org/docs/specifications/doubleratchet/>. [Accessed: April 22, 2026].
- [26] “Trying to understand Signal’s double ratchet protocol : r/cryptography - Reddit,” [Online]. Available: https://www.reddit.com/r/cryptography/comments/1ms1rr6/trying_to_understand_signals_double_ratchet/. [Accessed: April 22, 2026].
- [27] “Tauri Testing Guide | Claude Code Skill for Desktop Apps - MCP Market,” [Online]. Available: <https://mcpmarket.com/tools/skills/tauri-application-testing>. [Accessed: April 22, 2026].
- [28] “Tauri framework: Building lightweight desktop applications with Rust | by Kevin | CodeX,” [Online]. Available: <https://tianaschool.medium.com/tauri-framework-building-lightweight-desktop-applications-with-rust-3b3923c72e75>. [Accessed: April 22, 2026].
- [29] “Tests - Tauri,” [Online]. Available: <https://v2.tauri.app/develop/tests/>. [Accessed: April 22, 2026].
- [30] “An Efficient and Generic Construction for Signal’s Handshake (X3DH): Post-Quantum, State Leakage Secure, and Deniable,” [Online]. Available: <https://csrc.nist.gov/csrc/media/Events/2022/fourth-pqc-standardization->

conference/documents/papers/an-efficient-and-generic-construction-for-signals-handshake-pqc2022.pdf. [Accessed: April 22, 2026].